
MuLoCo: Muon is a Practical Inner Optimizer for DiLoCo

Benjamin Thérien^{1 2 3 *} Xiaolong Huang^{2 4} Aaron Defazio¹ Irina Rish^{2 3} Eugene Belilovsky^{2 4}

Abstract

DiLoCo is a powerful framework for training large language models (LLMs), enabling larger optimal batch sizes and increased accelerator utilization under networking constraints. However, DiLoCo’s performance has been shown to degrade as the number of workers (K) increases (Charles et al., 2025). In this work, we posit that a related but often overlooked factor in DiLoCo’s behavior is the choice of inner optimizer, which shapes the pseudogradient used by the outer optimizer. Given the recent success of Muon relative to AdamW for data parallel (DP) training, we examine how Muon’s normalized optimizer steps can affect the pseudogradient’s quality. We find that, relative to AdamW, Muon yields more *directionally correct* pseudogradients as the number of workers (K) increases. In our experiments pre-training language models, we conduct extensive hyperparameter tuning across 150M, 416M, 914M, 1.76B, and 3.1B models for DiLoCo, MuLoCo, AdamW DP, and Muon DP. Consistently across all scales, we find that with $K \geq 1$ workers, MuLoCo (Muon inner optimizer DiLoCo) achieves superior performance to DiLoCo in absolute terms and for $K > 2$ it outperforms DiLoCo relative to their data parallel baselines, while being compatible with quantization, streaming, and long synchronization intervals. At $K = 1$, we find that MuLoCo can even outperform the Muon DP gold standard while having larger critical batch sizes and faster training times as a result. A more detailed version of this manuscript and code is available on the project page <https://bentherien.github.io/muloco-1>.

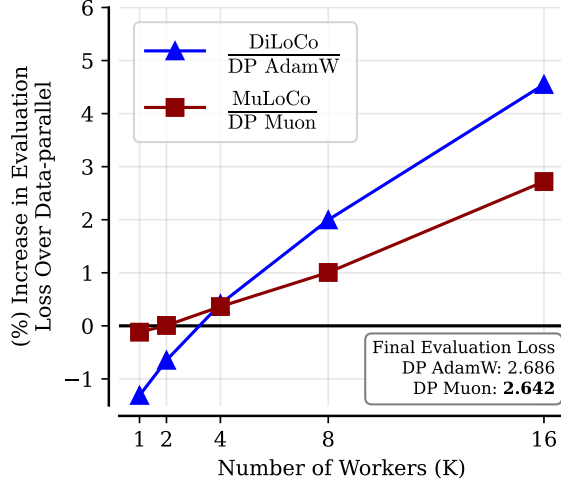
¹FAIR at Meta ²Mila ³Université de Montréal ⁴Concordia University. *Work done at Meta. Correspondence to: Benjamin Thérien <bentherien@umontreal.ca>.

1. Introduction

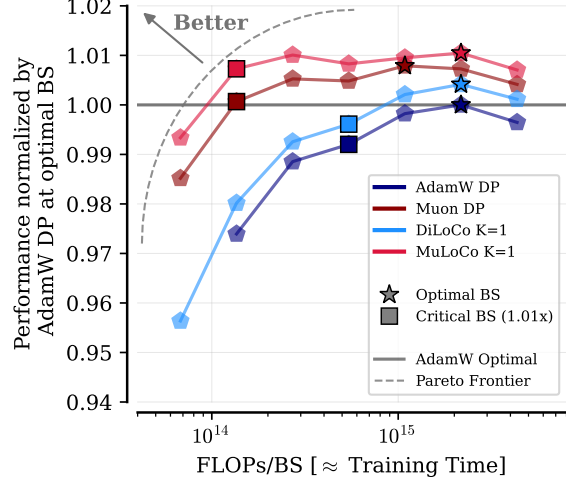
In Large Language Model (LLM) pre-training, it is now well established that scaling training data and model parameters in tandem at a ratio of roughly 20 tokens per parameter (TPP) is compute optimal (Hoffmann et al., 2022; Hägele et al., 2024; Besiroglu et al., 2024; Dey et al., 2023; Schaeffer et al., 2025). With the largest existing training runs reaching 10 – 40T tokens (Yang et al., 2025; Team et al., 2025; DeepSeek-AI, 2024) and estimates of available data reaching 300T (Villalobos et al., 2024), it is reasonable to assume that the largest compute optimal models trained in the near future will have 2-15T parameters. Efficiently training at this scale is bottlenecked by two requirements of data-parallel training: (a) the need to take a large number of gradient descent steps sequentially, leading to long wall-clock training times, and (b) the need to communicate among all model replicas at each optimization step, leading to decreased compute utilization across tens of thousands of accelerators. A straightforward remedy to (a) is training at large batch sizes. However, at the largest scales, this is likely to exacerbate (b) since there is a limit to the number of accelerators that are tightly interconnected in typical deployments (NVIDIA, 2024).

The motivation for training at large batch sizes is clear: leveraging more accelerators in parallel leads to faster wall-clock training times for the same amount of data. At the largest model scales, computing gradients on a single training example can be expensive. Without large batch training across enough accelerators in parallel, practitioners are required to (1) accumulate gradients, leading to longer step times, or (2) take more gradient descent steps sequentially. The critical batch size (CBS) (McCandlish et al., 2018; Merrill et al., 2025; Zhang et al., 2025; Bergsma et al., 2025) is known as the largest batch size that can be used to achieve performance nearly as strong as the optimal (best performing) batch size. Therefore, training at the CBS with enough devices in parallel can lead to linear speedups in training time with nearly no performance degradation, as long as communication costs do not grow with the number of devices. However, when training at the largest model scales or across poorly interconnected accelerators, this is rarely the case.

Due to the large communication costs associated with data-



(a) MuLoCo outperforms DiLoCo in absolute terms and continues to do so at larger worker counts even after normalizing by their respective data-parallel baselines.



(b) MuLoCo K=1 achieves a Pareto-Optimal Performance-Time Tradeoff. All datapoints are FLOP-matched.

Figure 1. **Subfigure (a)** reports the percentage increase in loss for DiLoCo and MuLoCo relative to their DP baselines. We observe that the relative performance of each method degrades as K is increased, but that MuLoCo shows clear improvements over DiLoCo relative to their data parallel baselines as workers are increased. **Subfigure (b)** reports the performance of different optimizers in an iso-FLOP setting as a function of FLOPs/Batch Size (a proxy for training time). The optimal batch size (B_{opt}) is the batch size achieving the best performance, while the critical batch size (B_{crit}) is taken to be the largest batch size achieving a final evaluation loss that is: $L(B_{\text{crit}}) \leq 1.01 \times L(B_{\text{opt}})$. We observe that at the 3.1B scale, our proposed MuLoCo K=1 achieves stronger performance than other optimizers while also being compatible with much larger batch sizes.

parallel (DP) training at large model scales or under bandwidth constraints, a number of communication-efficient training algorithms have been proposed. Most recently, DiLoCo (Douillard et al., 2023) has gained attention due to its competitive performance with DP pre-training of LLMs and its larger optimal batch sizes (Charles et al., 2025). DiLoCo independently trains a single model with DP AdamW across K workers in parallel while only communicating across workers every H gradient descent steps. When individual workers span highly interconnected compute pools (e.g., single nodes, racks, adjacent racks, or entire datacenters for the largest models) with significantly faster bandwidth within pools than across them, DiLoCo can lead to significant speedups relative to DP training across the same number of accelerators. However, Charles et al. (2025) shows that the performance of DiLoCo degrades as the number of workers is increased, leading to questions of its utility at large worker counts.

In this work, we seek to address DiLoCo’s worker scaling problem while simultaneously increasing its critical batch size. Our core hypothesis is that this degradation is related to the choice of inner optimizer, which shapes the pseudogradient used by the outer optimizer. Given the recent success of Muon relative to AdamW for LLM pre-training, its larger critical batch sizes, and its orthonormalized optimizers steps that are substantially different from those of AdamW (Jordan et al., 2024; Liu et al., 2025; Shah et al.,

2025a), we conjecture that, beyond its improved convergence, Muon may offer additional advantages in terms of compressibility, parallelization, and performance within a Local SGD framework. Given the potential of Muon to be a powerful inner optimizer for DiLoCo, in what follows, we systematically compare MuLoCo (Muon inner optimizer) to DiLoCo in settings using a large number of workers (K), long communication intervals (H), when using compressed communication, and when using streaming updates. We further scale compute following a chinchilla optimal prescription and examine whether our most interesting findings hold at up to 15B parameter scale. Our contributions can be summarized as follows:

- We establish that MuLoCo’s performance scales better with workers than DiLoCo. Specifically, beyond outperforming DiLoCo in absolute terms, MuLoCo outperforms DiLoCo at $K > 2$ workers even when normalizing by the performance of their respective data parallel baselines. We find that this advantage is maintained at scale.
- We demonstrate that MuLoCo outperforms DiLoCo under long synchronization intervals, under pseudogradient compression, with partitioned communication, and in wall-clock training time, while having lower memory complexity.
- At 3.1B scale, we find that MuLoCo’s critical batch

size is $2\text{--}4\times$ larger than DiLoCo’s, further reducing communication while increasing usable data parallelism.

- Empirically, we find that Muon’s orthonormalized steps lead to a more directionally aligned pseudogradient than AdamW and that the spectrum of AdamW’s individual weight differences collapses during averaging.

2. Background

We now briefly present the algorithmic details of the Muon optimizer and provide more detailed background in Sec. B of the appendix.

Muon is a recently proposed (Jordan et al., 2024) matrix-based optimizer that takes orthonormalized steps. Specifically, at each step, Muon updates its momentum accumulator,

$$m_t = \beta m_{t-1} + g_t,$$

and applies five steps of *Newton–Schulz* iteration (Björck & Bowie, 1971; Kovarik, 1970) to produce an orthogonalized update. Specifically, the iteration refines

$$X_{t,j} = p(X_{t,j-1} X_{t,j-1}^\top) X_{t,j-1},$$

where $p(x) = ax + bx^3 + cx^5$ with (a, b, c) empirically tuned to be $3.4445, -4.7750, 2.0315$ (Jordan et al., 2024). This procedure effectively scales and rotates m_t toward the nearest orthogonal matrix, approximating $UV^\top$ when $m_t = U\Sigma V^\top$ is its singular value decomposition. The resulting orthogonalized update O_t is then rescaled by the learning rate and applied as $W_t = W_{t-1} - \eta O_t - \eta \lambda W_{t-1}$ with optional decoupled weight decay. In this work, we always use decoupled weight decay with Muon as it was shown to be important in maintaining Muon’s convergence benefits at larger model sizes (Liu et al., 2025).

3. Method

Our main methodological contribution is to replace DiLoCo’s AdamW inner optimizer with Muon. While this is a simple change, our results (Sec. 5) will show that beyond improving over DiLoCo in absolute terms, MuLoCo improves performance at larger worker counts over DiLoCo *even when normalizing by data-parallel Muon and AdamW, respectively*. That is, MuLoCo’s improvements over DiLoCo **can not** entirely be explained by Muon’s performance improvement over AdamW. Our main assumption is that this improvement can be explained by Muon’s pseudogradients being more directionally correct than AdamW’s (e.g., as in Fig. 2). In this section, we introduce MuLoCo and theoretically analyse how the alignment of AdamW and Muon optimizer steps affects the pseudogradient’s nuclear

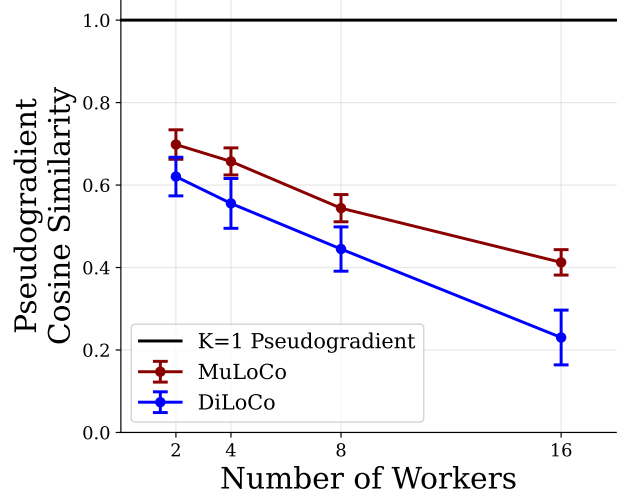


Figure 2. **Pseudogradient cosine similarities compared to a FLOP-matched $K = 1$ pseudogradient.** We report the mean cosine similarity across all hidden weight matrices of a 416M transformer, with error bars indicating variability across matrices. MuLoCo’s pseudogradients are consistently more aligned with the $K = 1$ pseudogradient than DiLoCo’s, and this alignment degrades more slowly as the number of workers increases.

norm, offering a speculative explanation for the observed performance difference between MuLoCo and DiLoCo.

3.1. MuLoCo

MuLoCo reduces communication by training multiple models in parallel and infrequently communicating weight-space parameter differences after H steps of local optimization. Specifically, each worker $k \in [K]$ samples data from its assigned shard D_k and performs local optimization as follows:

$$\theta_k^{(t+h)} = \text{Muon}(\theta_k^{(t+h-1)}, \nabla_{\theta} \mathcal{L}), \quad h = 1, \dots, H. \quad (1)$$

The worker-level parameter differences, $\Delta_k^{(t)} = \theta_k^{(t-H)} - \theta_k^{(t)}$, are then communicated to form a pseudogradient:

$$\Psi^{(t)} = \frac{1}{K} \sum_{k=1}^K \Delta_k^{(t)}, \quad (2)$$

which captures the average weight-space displacement across all workers’ inner optimization steps. The pseudogradient $\Psi^{(t)}$ is then used as input to SGD with Nesterov momentum, which updates its momentum and model weights as:

$$\begin{aligned} u^{(t)} &= \mu u^{(t-H)} + \eta_{\text{out}} \Psi^{(t)}, \\ \theta^{(t)} &= \theta^{(t-1)} - \mu u^{(t)} - \eta_{\text{out}} \Psi^{(t)}. \end{aligned} \quad (3)$$

It should be noted that DiLoCo can be recovered by swapping Muon for AdamW in equation 1.

3.2. Pseudogradient analysis

Since the weight decay hyperparameter for Muon and AdamW is generally chosen to be $\lambda \leq 10^{-1}$ and is additionally rescaled by the learning rate (α), the contribution of decoupled weight decay to the pseudogradient is negligible. As such, we exclude it from our analysis. We now demonstrate that the nuclear norm of general pseudogradients depends on the alignment of individual optimizer steps with the pseudogradient and on the Frobenius norm of the optimizer step, itself.

Proposition 3.1 (Pseudogradient nuclear norm depends on optimizer step alignment). *Fix $m, n \in \mathbb{N}$ and let $r := \min\{m, n\}$. Consider a pseudogradient of the form*

$$\Psi = \frac{1}{K} \sum_{k=1}^K \sum_{h=1}^H \alpha_{t+h} \psi^{(t+h,k)} \in \mathbb{R}^{m \times n},$$

where $\alpha_{t+h} \geq 0$ is the learning rate and $\psi^{(t+h,k)}$ is the optimizer step. Let $\Psi = U \Sigma V^\top$ be its singular value decomposition and define the orthonormal factor $\Psi^* := UV^\top$. Let the cosine similarity between a given optimizer step and Ψ^* be

$$\rho^{(t+h,k)} := \frac{\langle \psi^{(t+h,k)}, \Psi^* \rangle_F}{\|\psi^{(t+h,k)}\|_F \|\Psi^*\|_F}.$$

Then the nuclear norm of Ψ satisfies

$$\|\Psi\|_* = \frac{\sqrt{r}}{K} \sum_{k=1}^K \sum_{h=1}^H \rho^{(t+h,k)} \alpha_{t+h} \|\psi^{(t+h,k)}\|_F.$$

Proof. The proof is provided in section D of the appendix. $\square$.

It follows that $\|\psi^{(t+h,k)}\|_F = \sqrt{r}$ for Muon since $\psi^{(t+h,k)}$ is orthonormal. Therefore, MuLoCo’s pseudogradient’s nuclear norm naturally varies with the per-step alignment (ρ), the number of workers (K), and the inner learning rate (α). In contrast, the spectrum of AdamW’s pseudogradient also depends on $\|\psi^{(t+h,k)}\|_F$, which may be highly variable at higher worker counts, leading to overemphasizing certain steps unnecessarily. This exact behaviour is illustrated in Fig. 11. While it should be noted that our theory has no direct connection to performance and that all inferences made here are purely speculative, the slower decrease in cosine similarity of MuLoCo in Fig. 2 relative to DiLoCo is correlated with its relatively stronger performance at larger worker counts (Fig. 1). This leads us to hypothesize that the stronger alignment of MuLoCo’s optimizer steps (Fig. 10) and its stable Frobenius norm across them (Fig. 11) may lead to a larger pseudogradient spectrum with less interference during averaging, which may improve performance.

4. Empirical Evaluation

Base LM pre-training Task. For all experiments except scaling laws, we select a 416M dense transformer language modeling task (see 416M in Table 1 for exact model dimensions). The architecture follows Gemma3 (Kamath et al., 2025) with SwiGLU FFNs, QK norm, and additional RMS normalization layers before residual connections. We tokenize using the Llama3 tokenizer (Llama-Team, 2024). Unless otherwise specified, we synchronize every $H = 30$ local steps and use $K = 8$ workers. In all of our experiments, we use a sequence length of 2048, we decay the learning rate following a cosine decay schedule to $0.1 \times$ the maximum value, and we pre-train on the high-quality portion of the Nemotron-CC dataset (Su et al., 2024). For experiments scaling the communication steps H , with compressed updates, or using streaming DiLoCo, we fix the global batch size to $B = 1\text{M}$ tokens. In our scaling experiments, the batch size is swept as described below.

Scaling study. In our scaling experiments, we increase model and dataset size following a ratio of 20 tokens per parameter (TPP) (Hoffmann et al., 2022). We report exact model sizes and token budgets in Table 1. At each scale except 15B, we conduct extensive hyperparameter sweeps as described below.

Table 1. Gemma3-style transformers used in our scaling study. The architecture follows Gemma3 (Kamath et al., 2025) with SwiGLU FFNs, QK norm, and additional RMS normalization layers before residual connections. However, we tokenize using the Llama3 tokenizer (Llama-Team, 2024). All models use a sequence length of 2048.

Model Parameters	Transformer Layers	Attention Heads	QKV Dimension	Hidden Dimension	Token Budget	HP Sweep
150M	6	4	512	1,408	3B	✓
416M	12	8	1,024	2,816	8.16B	✓
914M	18	12	1,536	4,224	18.12B	✓
1.76B	24	16	2,048	5,632	35.23B	✓
3.07B	30	20	2,560	7,040	61.4B	✓
15.23B	54	36	4,608	12,672	304.6B	✗

Hyperparameter tuning. At all model scales, we tune the weight decay (λ), inner learning rate (η_{in}), global batch size (B), outer learning rate (η_{out}), and outer momentum (μ). Full details of the tuning procedure and search grids are provided in Appendix F.1.

DiLoCo We use AdamW as a baseline inner optimizer for DiLoCo. Following Charles et al. (2025) we set $\beta_1 = 0.9$ and $\beta_2 = 0.99$ for all experiments. Unless otherwise specified, we use a synchronization interval of $H = 30$.

MuLoCo We use the Muon with a quintic Newton-Shulz iteration (Jordan et al., 2024). Muon is applied to hidden layers, while AdamW is used for the embeddings, normalization, and output layers. For hidden matrices of size $W \in \mathbb{R}^{m \times n}$ we rescale the learning rate by $\sqrt{\frac{n}{m}}$. We set $\beta_1 = 0.9$ for both Muon and AdamW. We set AdamW’s

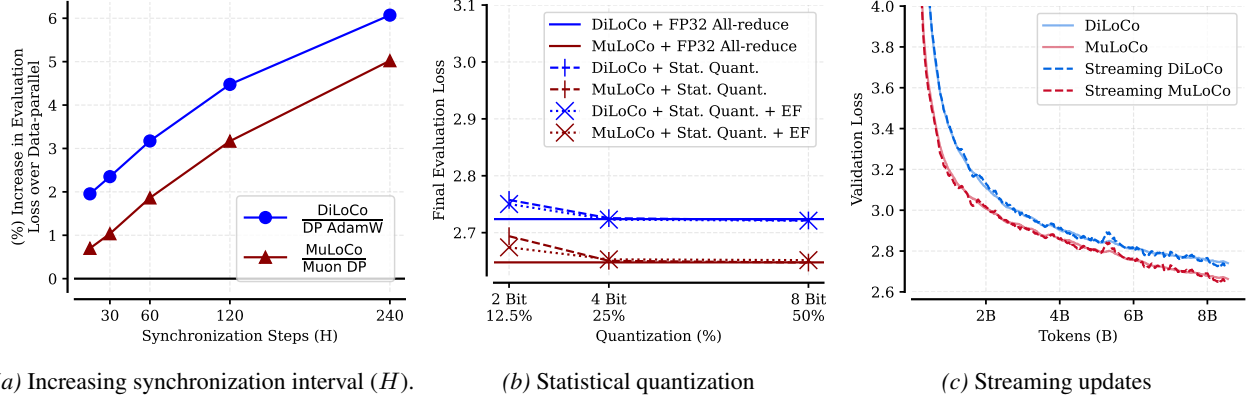


Figure 3. MuLoCo is compatible with longer sync. intervals, quantization, and streaming. (a) varies the synchronization interval H and shows that MuLoCo continues to outperform DiLoCo relative to their DP baselines, maintaining MuLoCo’s improvements at $K=8$ workers across different synchronization intervals. (b) Statistical quantization is shown with and without error feedback (EF) across bitwidths. MuLoCo outperforms DiLoCo at all bitwidths, and both methods are essentially lossless at 4 bits and above. Even at 2 bits, degradation is modest, with EF providing little additional benefit. (c) Streaming and non-streaming variants match closely for both DiLoCo and MuLoCo, indicating no degradation from partitioned (streamed) communication. In particular, MuLoCo tracks DiLoCo under streaming and remains fully compatible with streaming updates.

$\beta_2 = 0.99$. Unless otherwise specified, we use a synchronization interval of $H = 30$.

Outer Optimizer We use SGD with Nesterov momentum as the outer optimizer for all our DiLoCo and MuLoCo experiments with hyperparameters reported in Figure 23.

Data Parallel Baselines We also report the performance of well-tuned DP AdamW and DP Muon baselines.

Implementation For Muon, we use Ahn et al. (2025)’s distributed implementation. For AdamW, we used the fused PyTorch implementation. All our pre-training experiments are run using TorchTitan (Liang et al., 2025). We use the streaming DiLoCo implementation within TorchFT for all our distributed optimizers. We train across H100 and H200 GPUs. Our largest experiments at 15B scale use HSDP with 16 FSDP ranks and 64 data-parallel ranks. For DiLoCo and MuLoCo, they are accordingly subdivided among workers.

Evaluation Loss During training, we log the validation loss every 15 steps on 2M token batches from a held-out set of Netotron-CC pre-training data. Following (Hoffmann et al., 2022), to reduce stochasticity in the final loss estimate, we use a smoothed final loss. Further details are reported in Section G of the appendix.

5. Results—Ablations & Wall-Time

The following section compares the performance of MuLoCo and DiLoCo at 416M scale across a number of settings relevant to communication-efficient training.

MuLoCo vs. DiLoCo: Increasing K Figure 1 (a) reports the increase (lower is better) in final validation loss relative

to data-parallel as a function of the number of workers (k). It should be noted that on an absolute scale, MuLoCo strictly outperforms DiLoCo in each case due to Muon’s stronger performance. However, we also observe that both DiLoCo and MuLoCo outperform their respective data-parallel baselines for $K = 1$ and $K = 2$ workers. At these worker counts, DiLoCo’s improvements over data-parallel are larger than MuLoCo’s, suggesting that Muon’s curvature-aware steps may reduce the gains available from using an outer momentum. For $K > 2$ workers, we see a clear pattern emerge: *as the number of workers increases the performance of DiLoCo relative to its data parallel baseline decreases at a faster rate than MuLoCo does relative to Muon DP, demonstrating that MuLoCo’s performance scales more favourably as workers are increased.*

Finding 1: Relative to their data-parallel baselines, MuLoCo scales better than DiLoCo as the number of workers increases. (Fig. 1 (a))

Why does MuLoCo scale better with workers? Our central hypothesis is that Muon’s orthonormalized inner steps produce pseudogradients that remain more aligned with the data-parallel/ $K = 1$ pseudogradient as the number of workers grows. To test this, we train a 416M model with the optimal DP Muon and AdamW hyperparameters, resume saved checkpoints with DiLoCo or MuLoCo for $H = 30$ steps at $K \in \{2, 4, 8, 16\}$ while preserving optimizer states, save the resulting pseudogradients, and measure their cosine similarity to the corresponding DP/ $K = 1$ pseudogradient across vectorized hidden weights. Figure 8 shows that MuLoCo pseudogradients are consistently more directionally aligned than DiLoCo’s; Figure 10 further shows that the individual Muon steps forming a $K = 8$ pseudogradient

are more aligned with the final pseudogradient direction than AdamW steps, suggesting that Muon updates are naturally more mutually aligned. To understand DiLoCo’s degradation at larger K , we examine the spectra of worker weight differences Δ_k before and after averaging: Figure 9 shows that DiLoCo’s pseudogradient spectrum collapses much more than MuLoCo’s during averaging, and that this collapse worsens with worker count, matching DiLoCo’s strong performance at low-interference settings ($K = 1, 2$) and sharper degradation thereafter. While not causal, these results support our hypothesis that DiLoCo degrades primarily through information loss during averaging. Finally, although $K=1$ MuLoCo always outperforms $K=1$ DiLoCo in absolute terms, $K=1$ DiLoCo improves more relative to DP AdamW than $K=1$ MuLoCo does relative to DP Muon (Fig. 1a); we hypothesize this is because Muon steps are already well aligned, leaving less room for the outer optimizer, which is analogous to Reptile and encourages step alignment (Nichol et al., 2018), to improve over DP Muon than it can over DP AdamW.

MuLoCo vs. DiLoCo: Increasing H In this section, we study the performance of MuLoCo and DiLoCo relative to their data-parallel baselines as the synchronization interval (H) is progressively doubled from $H = 15$ to $H = 240$. To accomplish this, we train DP baselines in our chinchilla-optimal 8B token setting for 8160 steps at batch size 512 and sequence length 2048. For DiLoCo and MuLoCo, we fix the number of workers at $K = 8$ and vary H . Figure 3 (b) reports the performance of both methods relative to data-parallel. We observe that for all synchronization intervals tested, MuLoCo outperforms DiLoCo relative to their respective data-parallel baselines. Furthermore, the improvements of MuLoCo over DiLoCo remain mostly constant across H values, showing that the communication frequency no effect on performance differences between DiLoCo and MuLoCo relative to DP.

Finding 2: MuLoCo maintains a *relative* performance advantage over DiLoCo as the synchronization interval is increased. (Fig. 3 (b))

MuLoCo vs. DiLoCo: Quantization Figure 13 reports the final evaluation loss of models trained using DiLoCo and MuLoCo with 8-bit, 4-bit, and 2-bit quantization in a $K=8$, $H=30$ setting. We experiment with global linear quantization (a) and statistical quantization (b) (also in Fig 3 (b)). Our experiments assume that communication is performed using an efficient all-to-all reduce-scatter (Wang et al., 2023a) and subsequent all-gather to avoid compounding quantization errors incurred by a ring all-reduce with many workers. We observe that MuLoCo outperforms DiLoCo in absolute terms and that both achieve lossless compression up to 4-bits. When compressing to 2-bits there is a substantial performance degradation for Linear quantization but not

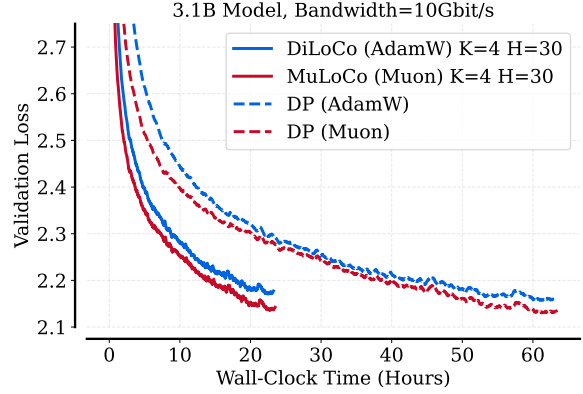


Figure 4. **Wall-clock and system-level performance.** **Top:** Under 10 Gbit/s bandwidth, MuLoCo achieves lower loss at any training duration despite DiLoCo’s slightly lower per-step overhead; both communication-efficient methods substantially outperform DP baselines in time-to-loss. **Bottom:** System metrics (median over first 500 steps). Configuration: 3.1B model, 2M token batch (1024×2048), $K = 4$ workers, $H = 30$, 4×8 H100 GPUs. MuLoCo has negligible overhead ($< 1\%$) relative to DiLoCo.

statistical. In Figure 13 we report additional experiments where quantization is applied individually to each row of a tensor. We find that this improves the performance of linear quantization at 2-bits and, in the statistical case, has near identical performance to global quantization.

Finding 3: Both MuLoCo and DiLoCo achieve lossless 4-bit quantization. (Fig. 3 (b))

MuLoCo vs. DiLoCo: Top- k Sparsification Figure 14 reports the performance of MuLoCo and DiLoCo with and without error-feedback under top- k sparsification retaining 0.5%, 1%, 2.5%, 5%, 10%, 20%, and 50% of entries. We observe that MuLoCo consistently outperforms DiLoCo and that Error feedback helps prevent performance degradation. However, the degradation in performance can be substantial at larger sparsities. Given the additional overhead of communicating the index, the true compression ratios of vanilla top- k sparsification are higher than the reported sparsity, making the more competitive sparsity ratios of 5-10% less appealing than using 2-bit quantization with an all-to-all reduce scatter. Nevertheless, if very strong compression is the goal, practitioners can significantly benefit from using MuLoCo with error feedback over DiLoCo.

Streaming MuLoCo In this section, we investigate Mu-

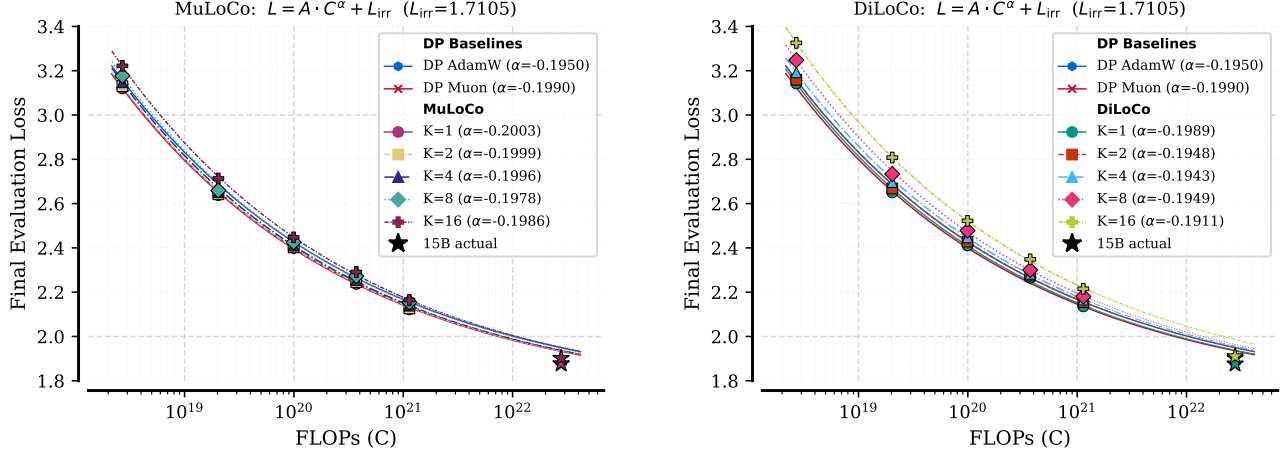


Figure 5. **Power-law fits in compute with a shared irreducible loss.** We fit power laws of the form $L(C) = AC^\alpha + L_{\text{irr}}$ to the final evaluation loss as a function of training compute C , using a *joint* (shared) irreducible loss L_{irr} across methods. The fitted exponents α are larger for MuLoCo than DiLoCo, indicating better performance with scale. We also find that losses at 15B are underestimated.

LoCo’s performance when using partitioned communication (Douillard et al., 2025). Specifically, in our $K=8$ worker and $H=30$ communication step setting, we partition the model’s parameters into thirds and take outer steps for partitions every 10 steps. This has the effect of reducing the peak communication volume by a factor equal to the number of partitions. Figure 3 (c) reports validation loss curves during training for DiLoCo, MuLoCo, and their streaming counterparts. We observe that both the streaming and non-streaming methods reach the same final validation loss, demonstrating that MuLoCo is compatible with streaming.

Finding 4: MuLoCo is compatible with the partitioned communication of streaming DiLoCo.

Wall-clock training time In this section, we (1) validate the practicality of using Muon as the inner optimizer for DiLoCo, given its more expensive optimizer step, and (2) illustrate the utility of MuLoCo for training under bandwidth constraints. Using the distributed implementation of (Ahn et al., 2025) in combination with HSDP within TorchTitan (Liang et al., 2025), we train a 3.1B model across four $8 \times H100$ nodes using FSDP within nodes and data parallelism across nodes. Table 4 reports the end-to-end step time, MFU, TFLOPS/GPU, throughput, and training time in a chinchilla optimal regime. We find that the optimizer has negligible training time overhead: +0.96%. In figure 4 we report wall-clock training curves, demonstrating that Muon’s overhead is negligible (MuLoCo dominates DiLoCo) and showing the significant benefits of using MuLoCo over Muon DP in a bandwidth-constrained environment with 10 Gbit/s networks (23 v.s. 60+ hours).

Finding 5: MuLoCo has negligible wall-clock overhead relative to DiLoCo. (Tab. 4)

6. Results—Scaling up

In this section, we investigate whether MuLoCo’s performance improvements over DiLoCo hold as model and dataset size are increased under a chinchilla optimal prescription (Hoffmann et al., 2022). We extensively tune MuLoCo, DiLoCo, and their respective DP baselines on 150M-3.1B language model pre-training tasks (see Tab. 1) and fit power laws to their final evaluation loss. We use a communication interval of $H = 30$ steps for these experiments and do not use any compression or streaming.

Fitting scaling laws. We first evaluate the fit of three candidate compute scaling laws: (i) $L(C) = aC^\alpha$, a simple power law; (ii) $L(C) = aC^\alpha + c$, with a per-optimizer irreducible loss; and (iii) $L(C) = aC^\alpha + L_0$, with a joint irreducible loss L_0 shared across all 12 optimizer/worker combinations while each retains independent a and b . Fitting hyperparameters are given in Sec E.2.

Table 2. **Comparison of compute scaling law forms.** We hold out the 3.1B scale from fitting for each power law. We report the mean absolute log-space residual across all 12 optimizer/worker combinations. We find that adding an irreducible loss substantially extrapolates to unseen scales.

Functional Form	Train Residual 150M–1.7B	Evaluation Residual	
		3.1B	15B
$L(C) = AC^\alpha$	0.013	0.056	0.160
$L(C) = AC^\alpha + c$	0.001	0.011	0.053
$L(C) = AC^\alpha + L_{\text{irr}}$	0.001	0.011	0.057

Evaluating functional forms for $L(C)$. Table 2 reports residuals from the different scaling laws fit to 150M – 1.7B datapoints. We hold out 3.1B scale to evaluate the performance of each method. We find that adding an irreducible loss substantially improves extrapolation, reducing the mean 3.1B residual from 0.056 to 0.011. Turning to the 15B

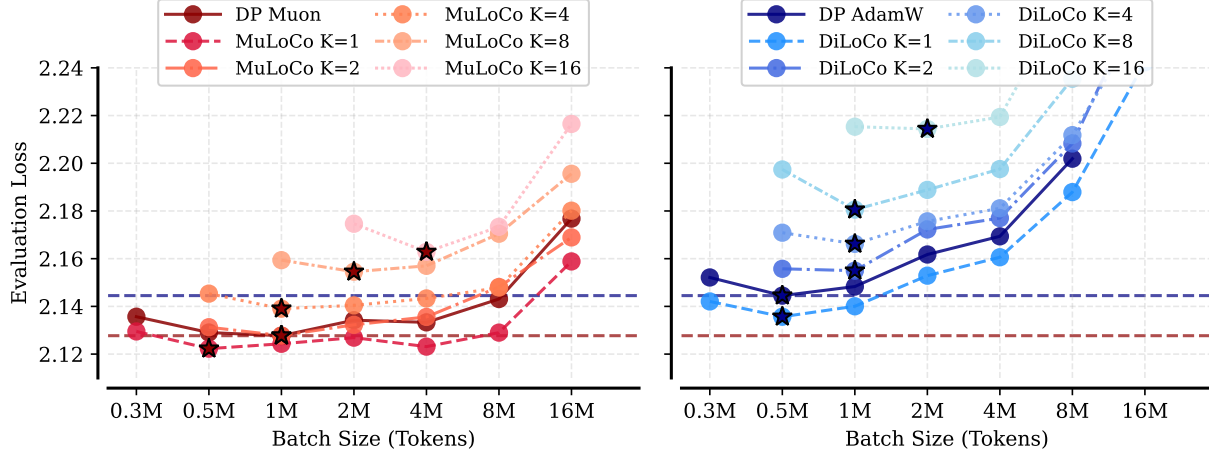


Figure 6. **MuLoCo has much larger critical batch sizes than DiLoCo.** For 3.1B models, we tune each method separately at every batch size and report final evaluation loss. MuLoCo maintains strong performance over a much wider batch-size range, enabling more parallelizable training, while DiLoCo degrades rapidly at large batch sizes.

residuals, we observe that using an irreducible loss also significantly improves extrapolation at this scale. Given this stronger extrapolation and the lower parameter count of a shared irreducible loss we opt to use this method.

Final $L(C)$. We now re-fit the power law with a joint irreducible loss to all five training scales (150M-3.1B). Table 5 of the appendix reports the fitted parameters, while Figure 5 plots the corresponding curves. We find that an irreducible loss of $L_{irr} = 1.711$ achieves the best fit. The scaling exponents cluster around $\alpha \approx -0.20$ ($\alpha \in [-0.200, -0.191]$), with MuLoCo having generally larger α values than DiLoCo across the board, indicating stronger scaling with compute. When evaluating the extrapolation to 15B we observe that the power laws consistently underreport performance at larger scale, suggesting that an irreducible loss of $L_{irr} = 1.711$ is likely too large in practice. Given this finding, in figure 16 of the appendix, we fit compute scaling laws at fixed L_{irr} values shared across all methods. For each L_{irr} , we report the resulting scaling exponent α normalized by the exponent of the corresponding DP baseline. We find that at lower irreducible losses, which are likely more realistic than $L_{irr} = 1.711$, MuLoCo at higher worker counts is projected to match and eventually outperform the data-parallel baseline.

Finding 6: The scaling exponents of Muon-based methods are smaller, demonstrating their stronger scaling.

Performance at different worker counts. Figure 18 reports the performance of MuLoCo and DiLoCo normalized by their respective data parallel baselines as a function of model size. First, we find that performance improvements of MuLoCo over DiLoCo, even when normalized by their respective data-parallel baselines, hold at larger scales. Second, we interpret our results within the context of Charles et al. (2025) who found that, at all worker counts

(K), DiLoCo follows a trend of strict performance improvement relative to DP AdamW as model size is increased. In our experiments, we find the same trend for $K=16$ MuLoCo and $K=16$ DiLoCo, suggesting that these optimizers should continue to improve relative to data parallel with scale. However, we don’t observe the same strict trend of monotonic performance improvement across all model sizes as in Charles et al. (2025). While we are uncertain about the exact cause of this discrepancy, we conjecture that it is caused by a small experimental difference.

Finding 7: At $K=16$ MuLoCo consistently improves with scale relative to the DP Muon.

Finding 8: MuLoCo’s improved performance over DiLoCo at $K > 2$ even when normalized by their respective data-parallel baselines is maintained at scale.

6.1. On MuLoCo’s Large Critical Batch Size

Critical batch size at 3.1B. Figure 6 reports the final evaluation loss of 3.1B models tuned at each batch size. We observe that AdamW degrades substantially as batch size increases, while MuLoCo maintains strong performance relative to its optimal batch size until $\sim 8M$ tokens at all worker counts. This gives MuLoCo increasingly parallelizable training and places $K=1$ MuLoCo on the Pareto frontier of performance versus training time: it achieves stronger final loss than AdamW DP, Muon DP, and $K=1$ DiLoCo while supporting the largest critical batch size. At the largest scales, fully exploiting this parallelism will likely require training across datacenters, leading to communication bottlenecks for DP Muon, but **not** for MuLoCo at $K>1$. Even when communication is not a bottleneck, MuLoCo at $K=1$ offers a notable advantage over Muon DP, especially at $B = 8M$ where it matches the optimal performance of Muon DP, achieved at a much smaller $B_{opt} = 1M$.

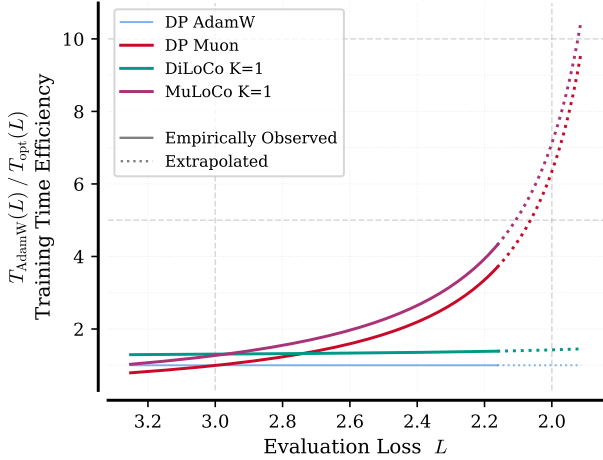


Figure 7. MuLoCo achieves substantial training-time gains at iso-loss. We report iso-loss training-time efficiency relative to DP AdamW. The ratio $T_{\text{AdamW}}(L)/T_{\text{opt}}(L)$ measures how many more sequential FLOPs AdamW requires to reach the same loss L , assuming sufficient parallel and well-interconnected accelerators to realize speedups from larger critical batch sizes. At observed scales, MuLoCo reaches the same loss as AdamW in roughly $\sim 4.5\times$ fewer sequential FLOPs. Dotted curves indicate extrapolation beyond the range of available data.

Finding 9: MuLoCo’s critical batch size at 3.1B scale is larger than DiLoCo’s at all worker counts (K), and $K=1$ MuLoCo lies on the Pareto frontier between final performance and training time (FLOPs/BS).

Training-time efficiency For large-scale pre-training, wall-clock time is often governed by the number of sequential optimization steps, making the critical batch size (B_{crit}) central to practical efficiency. We therefore compare optimizers using the training-time proxy $T_m(L) = C_m(L)/B_{\text{crit}}^{(m)}(C_m(L))$, where $C_m(L)$ is obtained by inverting the fitted loss law $L_m(C) = a_m C^{b_m} + L_0$ and $B_{\text{crit}}^{(m)}(D) = A_m D^{\alpha_m}$ is the fitted critical-batch-size law. The resulting iso-loss training-time ratio decomposes as

$$\frac{T_{\text{AdamW}}(L)}{T_{\text{opt}}(L)} = \underbrace{\frac{C_{\text{AdamW}}(L)}{C_{\text{opt}}(L)}}_{\text{compute savings}} \times \underbrace{\frac{B_{\text{crit}}^{(m)}(C_m(L))}{B_{\text{crit}}^{(\text{AdamW})}(C_{\text{AdamW}}(L))}}_{\text{parallelism advantage}}, \quad (4)$$

so values above one indicate that optimizer m reaches the same loss as DP AdamW in fewer sequential FLOPs.

Figure 19 shows that $K=1$ MuLoCo has the strongest iso-loss training-time advantage, reaching the same loss as DP AdamW in roughly $\sim 4.5\times$ fewer sequential steps at observed scales. This gain is larger than MuLoCo’s compute-only advantage because it combines stronger loss scaling with a substantially larger critical batch size. We exclude MuLoCo and DiLoCo at $K \neq 1$ here as their batch sizes were not sufficiently swept to determine B_{crit} at each scale.

Finding 10: At iso-loss, $K=1$ MuLoCo reaches the same loss as DP AdamW in roughly $\sim 4.5\times$ fewer sequential steps at observed scales.

Scaling to 15B We use the scaling trends from our 150M–3.1B sweeps to extrapolate hyperparameters to 15B scale, then train six models: DP AdamW, DP Muon, DiLoCo, and MuLoCo, with the distributed methods evaluated at $K=1$ and $K=16$. These experiments are not fully swept at 15B, but they test whether the trends observed at smaller scales remain predictive for large-scale training. We find that MuLoCo $K=1$ matches the best downstream evaluation performance while using a much larger 16M-token batch size, substantially reducing the number of sequential optimization steps. At $K=16$, MuLoCo nearly matches single-worker performance at this scale, supporting the view that its worker-scaling behavior improves with model scale. Full hyperparameter details, evaluation results, and wall-clock analyses are provided in Appendix F.2.

7. Conclusion

This work compares the performance of DiLoCo to MuLoCo across a range of workloads relevant to distributed optimization and as training is scaled up. We established that MuLoCo outperforms DiLoCo under long communication intervals, compressed communication, when using streaming updates, and even in wall-clock training time. Beyond improving over DiLoCo in absolute terms, we even found that MuLoCo’s performance relative to DP Muon scales better than DiLoCo’s performance relative to DP AdamW as the number of workers increases. Furthermore, we found that MuLoCo’s critical batch size is larger than DiLoCo’s at all worker counts, leading to faster wall-clock training times and possibly warranting training across data-centers to take advantage of the additional data parallelism. Finally, we found that $K=1$ MuLoCo has a Pareto-optimal performance-training-time tradeoff among all optimizers in our study. Overall, our results demonstrate that MuLoCo is a flexible distributed optimization algorithm. With its large critical batch sizes, stronger performance than DP Muon at $K=1$, and its improved worker scaling over DiLoCo, MuLoCo has significant advantages for large-scale training in both low-bandwidth and high-bandwidth settings.

Limitations. While we aim to provide a sound empirical study, our evaluation is necessarily limited in scope. In particular, we only study 150M–15B dense transformers trained in a 20TPP setting, so the predicted scaling trends may not hold several orders of magnitude beyond this range or for other architectures such as MoEs, SSMs, or hybrid models. We also leave the study of alternative outer optimizers beyond Nesterov SGD and convergence guarantees for future work.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgements

We are particularly grateful to Lin Xiao for providing feedback on our manuscript and guidance on our experiments throughout. We thank Jack Rae and Tom Gunter for providing practical guidance regarding our scaling law and critical batch size experiments. We thank Andrei Mircea and Ekaterina Lobacheva for insightful discussions about our pseudogradient analysis experiments. We thank Howard Huang and Tushar Jain for their advice in resolving TorchTitan and TorchFT bugs. We would also like to thank Zeyuan Allen-Zhu, Konstantin Mishchenko, Ashok Cutkosky, Ziqui Bu, Alice Yang, Lisa jin, Andrey Gromov, Maissam Barkeshli, and Sanae Lotfi for fruitful comments and research discussions.

We acknowledge support FRQNT New Scholar [E.B.], the FRQNT Doctoral (B2X) scholarship [B.T.], the Canada CIFAR AI Chair Program [I.R.], and the Canada Excellence Research Chairs Program in Autonomous AI [I.R.].

References

- Abbes, I., Subbaraj, G., Riemer, M., Islah, N., Thérien, B., Tabaru, T., Kingetsu, H., Chandar, S., and Rish, I. Revisiting replay and gradient alignment for continual pre-training of large language models. *CoRR*, abs/2508.01908, 2025. URL <https://doi.org/10.48550/arXiv.2508.01908>.
- Ahn, K., Xu, B., Abreu, N., Fan, Y., Magakyan, G., Sharma, P., Zhan, Z., and Langford, J. Dion: Distributed orthonormalized updates, 2025. URL <https://arxiv.org/abs/2504.05295>.
- Alistarh, D., Grubic, D., Li, J., Tomioka, R., and Vojnovic, M. Qsgd: Communication-efficient sgd via gradient quantization and encoding. *Advances in neural information processing systems*, 30, 2017.
- Basu, D., Data, D., Karakus, C., and Diggavi, S. Qsparse-local-sgd: Distributed sgd with quantization, sparsification, and local computations, 2019.
- Bergsma, S., Dey, N. S., Gosal, G., Gray, G., Soboleva, D., and Hestness, J. Power lines: Scaling laws for weight decay and batch size in LLM pre-training. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=bFXbLQzRoZ>.
- Besiroglu, T., Erdil, E., Barnett, M., and You, J. Chinchilla scaling: A replication attempt. *arXiv preprint arXiv:2404.10102*, 2024. URL <https://arxiv.org/abs/2404.10102>.
- Björck, Å. and Bowie, C. An iterative algorithm for computing the best estimate of an orthogonal matrix. *SIAM Journal on Numerical Analysis*, 1971. Cited on pages 1 and 8.
- Charles, Z., Teston, G., Dery, L., Rush, K., Fallen, N., Garrett, Z., Szlam, A., and Douillard, A. Communication-efficient language model training scales reliably and robustly: Scaling laws for diloco, March 2025. URL <https://arxiv.org/abs/2503.09799>.
- Cheng, Z. and Glasgow, M. Convergence of distributed adaptive optimization with local updates. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=VNq7srnvD9>.
- DeepSeek-AI. Deepseek-v3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- Defazio, A., Mishchenko, K., Raman, P., Shi, H. M., and Xiao, L. Smoothing diloco with primal averaging for faster training of llms. *CoRR*, abs/2512.17131, 2025. URL <https://doi.org/10.48550/arXiv.2512.17131>.
- Dey, N., Gosal, G., Chen, Z., Khachane, H., Marshall, W., Pathria, R., Tom, M., and Hestness, J. Cerebras-gpt: Open compute-optimal language models trained on the cerebras wafer-scale cluster. *arXiv preprint arXiv:2304.03208*, 2023. URL <https://arxiv.org/abs/2304.03208>.
- Douillard, A., Feng, Q., Rusu, A. A., Chhaparia, R., Donchev, Y., Kuncoro, A., Ranzato, M., Szlam, A., and Shen, J. Diloco: Distributed low-communication training of language models. *arXiv preprint arXiv:2311.08105*, 2023.
- Douillard, A., Donchev, Y., Rush, K., Kale, S., Charles, Z., Garrett, Z., Teston, G., Lacey, D., McIlroy, R., Shen, J., Ramé, A., Szlam, A., Ranzato, M., and Barham, P. Streaming diloco with overlapping communication: Towards a distributed free lunch, January 2025. URL <https://arxiv.org/abs/2501.18512>.
- Goyal, P., Dollár, P., Girshick, R. B., Noordhuis, P., Wesolowski, L., Kyrola, A., Tulloch, A., Jia, Y., and He, K. Accurate, large minibatch SGD: training imagenet in 1 hour. *CoRR*, abs/1706.02677, 2017. URL <http://arxiv.org/abs/1706.02677>.

- Hägele, A., Bakouch, E., Kosson, A., Ben Allal, L., Von Werra, L., and Jaggi, M. Scaling laws and compute-optimal training beyond fixed training durations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. doi: 10.52202/079017-2427.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. Training compute-optimal large language models. *CoRR*, abs/2203.15556, 2022. URL <https://doi.org/10.48550/arXiv.2203.15556>.
- Huber, P. J. Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics*, 35(1):73 – 101, 1964. doi: 10.1214/aoms/1177703732. URL <https://doi.org/10.1214/aoms/1177703732>.
- Iacob, A., Jovanovic, A., Safaryan, M., Kurmanji, M., Sani, L., Horváth, S., Shen, W. F., Qiu, X., and Lane, N. D. MT-DAO: Multi-timescale distributed adaptive optimizers with local updates. In *The Fourteenth International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=5yPP238v4c>.
- Iacob, A., Sani, L., Safaryan, M., Giampouras, P., Horváth, S., Jovanovic, A., Kurmanji, M., Aleksandrov, P., Shen, W. F., Qiu, X., and Lane, N. D. DES-LOC: Desynced low communication adaptive optimizers for foundation models. In *The Fourteenth International Conference on Learning Representations*, 2026b. URL <https://openreview.net/forum?id=6N2qFixxYZ>.
- Jaghoular, S., Ong, J. M., Basra, M., Obeid, F., Straube, J., Keiblinger, M., Bakouch, E., Atkins, L., Panahi, M., Goddard, C., Ryabinin, M., and Hagemann, J. INTELLECT-1 technical report. *CoRR*, abs/2412.01152, 2024a. URL <https://doi.org/10.48550/arXiv.2412.01152>.
- Jaghoular, S., Ong, J. M., and Hagemann, J. Opendiloco: An open-source framework for globally distributed low-communication training, July 2024b. URL <https://arxiv.org/abs/2407.07852>.
- Jastrzebski, S., Kenton, Z., Arpit, D., Ballas, N., Fischer, A., Bengio, Y., and Storkey, A. J. Three factors influencing minima in SGD. *CoRR*, abs/1711.04623, 2017. URL <http://arxiv.org/abs/1711.04623>.
- Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://web.archive.org/web/20250122060345/https://kellerjordan.github.io/posts/muon/>.
- Joseph, C.-É., Thérien, B., Moudgil, A., Knyazev, B., and Belilovsky, E. Meta-learning optimizers for communication-efficient learning. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=uRbf9ANAns>.
- Jovanović, A., Iacob, A., Safaryan, M., Modoranu, I.-V., Sani, L., Shen, W. F., Qiu, X., Alistarh, D., and Lane, N. D. Lordo: Distributed low-rank optimization with infrequent communication, 2026. URL <https://arxiv.org/abs/2602.04396>.
- Kallusky, D., Rao, V., Nandavanam, V., and Shi, H. M. SNOO: step-k nesterov outer optimizer - the surprising effectiveness of nesterov momentum applied to pseudo-gradients. *CoRR*, abs/2510.15830, 2025. URL <https://doi.org/10.48550/arXiv.2510.15830>.
- Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., Rouillard, L., Mesnard, T., Cideron, G., Grill, J., Ramos, S., Yvinec, E., Casbon, M., Pot, E., Penchev, I., Liu, G., Visin, F., Kenealy, K., Beyer, L., Zhai, X., Tsitsulin, A., Busa-Fekete, R., Feng, A., Sachdeva, N., Coleman, B., Gao, Y., Mustafa, B., Barr, I., Parisotto, E., Tian, D., Eyal, M., Cherry, C., Peter, J., Sinopalnikov, D., Bhupatiraju, S., Agarwal, R., Kazemi, M., Malkin, D., Kumar, R., Vilar, D., Brusilovsky, I., Luo, J., Steiner, A., Friesen, A., Sharma, A., Sharma, A., Gilady, A. M., Goedeckemeyer, A., Saade, A., Kolesnikov, A., Bende-bury, A., Abdagic, A., Vadi, A., György, A., Pinto, A. S., Das, A., Bapna, A., Miech, A., Yang, A., Paterson, A., Shenoy, A., Chakrabarti, A., Piot, B., Wu, B., Shahriari, B., Petrini, B., Chen, C., Lan, C. L., Choquette-Choo, C. A., Carey, C., Brick, C., Deutsch, D., Eisenbud, D., Cattle, D., Cheng, D., Paparas, D., Sreepathihalli, D. S., Reid, D., Tran, D., Zelle, D., Noland, E., Huizenga, E., Kharitonov, E., Liu, F., Amirkhanyan, G., Cameron, G., Hashemi, H., Klimczak-Plucinska, H., Singh, H., Mehta, H., Lehri, H. T., Hazimeh, H., Ballantyne, I., Szpektor, I., Nardini, I., Pouget-Abadie, J., Chan, J., Stanton, J., Wieting, J., Lai, J., Orbay, J., Fernandez, J., Newlan, J., Ji, J., Singh, J., Black, K., Yu, K., Hui, K., Vodrahalli, K., Greff, K., Qiu, L., Valentine, M., Coelho, M., Ritter, M., Hoffman, M., Watson, M., Chaturvedi, M., Moynihan, M., Ma, M., Babar, N., Noy, N., Byrd, N., Roy, N., Momchev, N., Chauhan, N., Bunyan, O., Botarda, P., Caron, P., Rubenstein, P. K., Culliton, P., Schmid, P., Sessa, P. G., Xu, P., Stanczyk, P., Tafti, P., Shivanna, R., Wu, R., Pan, R., Rokni, R., Willoughby, R., Vallu, R., Mullins, R., Jerome, S., Smoot, S., Girgin, S., Iqbal,

- S., Reddy, S., Sheth, S., Pöder, S., Bhatnagar, S., Panyam, S. R., Eiger, S., Zhang, S., Liu, T., Yacovone, T., Liechty, T., Kalra, U., Evci, U., Misra, V., Roseberry, V., Feinberg, V., Kolesnikov, V., Han, W., Kwon, W., Chen, X., Chow, Y., Zhu, Y., Wei, Z., Egyed, Z., Cotruta, V., Giang, M., Kirk, P., Rao, A., Lo, J., Moreira, E., Martins, L. G., Sanseviero, O., Gonzalez, L., Gleicher, Z., Warkentin, T., Mirrokni, V., Senter, E., Collins, E., Barral, J. K., Ghahramani, Z., Hadsell, R., Matias, Y., Sculley, D., Petrov, S., Fiedel, N., Shazeer, N., Vinyals, O., Dean, J., Hassabis, D., Kavukcuoglu, K., Farabet, C., Buchatskaya, E., Alayrac, J., Anil, R., Lepikhin, D. D., Borgeaud, S., Bachem, O., Joulin, A., Andreev, A., Hardin, C., Dadashi, R., and Hussenot, L. Gemma 3 technical report. *CoRR*, abs/2503.19786, 2025. URL <https://doi.org/10.48550/arXiv.2503.19786>.
- Karimireddy, S. P., Rebjock, Q., Stich, S. U., and Jaggi, M. Error feedback fixes signsgd and other gradient compression schemes. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 3252–3261, 2019. URL <https://arxiv.org/abs/1901.09847>.
- Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. On large-batch training for deep learning: Generalization gap and sharp minima. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. URL <https://openreview.net/forum?id=HloyRlYgg>.
- Kovarik, Z. Some iterative methods for improving orthonormality. *SIAM Journal on Numerical Analysis*, 1970. Cited on pages 1 and 8.
- Liang, W., Liu, T., Wright, L., Constable, W., Gu, A., Huang, C.-C., Zhang, I., Feng, W., Huang, H., Wang, J., Purandare, S., Nadathur, G., and Idreos, S. TorchTitan: One-stop pytorch native solution for production ready LLM pretraining. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=SFN6Wm7YBI>.
- Lin, T., Stich, S. U., and Jaggi, M. Don’t use large mini-batches, use local SGD. *CoRR*, abs/1808.07217, 2018.
- Lin, T., Kong, L., Stich, S. U., and Jaggi, M. Extrapolation for large-batch training in deep learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pp. 6059–6069. PMLR, 2020.
- Liu, J., Su, J., Yao, X., Jiang, Z., Lai, G., Du, Y., Qin, Y., Xu, W., Lu, E., Yan, J., Chen, Y., Zheng, H., Liu, Y., Liu, S., Yin, B., He, W., Zhu, H., Wang, Y., Wang, J., Dong, M., Zhang, Z., Kang, Y., Zhang, H., Xu, X., Zhang, Y., Wu, Y., Zhou, X., and Yang, Z. Muon is scalable for llm training, February 2025. URL <https://arxiv.org/abs/2502.16982>.
- Llama-Team. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- McCandlish, S., Kaplan, J., Amodei, D., and Team, O. D. An empirical model of large-batch training. *CoRR*, abs/1812.06162, 2018. URL <http://arxiv.org/abs/1812.06162>.
- McMahan, B., Moore, E., Ramage, D., Hampson, S., and y Arcas, B. A. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pp. 1273–1282. PMLR, 2017.
- Merrill, W., Arora, S., Groeneveld, D., and Hajishirzi, H. Critical batch size revisited: A simple empirical approach to large-batch language model training. *CoRR*, abs/2505.23971, 2025. URL <https://doi.org/10.48550/arXiv.2505.23971>.
- Nichol, A., Achiam, J., and Schulman, J. On first-order meta-learning algorithms. *CoRR*, abs/1803.02999, 2018. URL <http://arxiv.org/abs/1803.02999>.
- Nocedal, J. Updating quasi-newton matrices with limited storage. *Mathematics of Computation*, 35(151):773–782, 1980. ISSN 00255718, 10886842. URL <http://www.jstor.org/stable/2006193>.
- NVIDIA. *NVIDIA DGX SuperPOD: Next-Generation Scalable Infrastructure for AI Leadership Reference Architecture*. NVIDIA Corporation, 2024. Reference Architecture RA-11333-001 V11.
- Peng, B., Quesnelle, J., and Kingma, D. P. Decoupled momentum optimization. *arXiv preprint arXiv:2411.19870*, 2024.
- Reddi, S. J., Charles, Z., Zaheer, M., Garrett, Z., Rush, K., Konečný, J., Kumar, S., and McMahan, H. B. Adaptive federated optimization. In *Proceedings of the 9th International Conference on Learning Representations (ICLR)*, 2021. URL <https://arxiv.org/abs/2003.00295>.
- Rothchild, D., Panda, A., Ullah, E., Ivkin, N., Stoica, I., Braverman, V., Gonzalez, J., and Arora, R. Fetchsgd: Communication-efficient federated learning with sketching. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pp. 8253–8265. PMLR, 2020. URL <http://proceedings.mlr.press/v119/rothchild20a.html>.

- Sani, L., Iacob, A., Cao, Z., Marino, B., Gao, Y., Paulik, T., Zhao, W., Shen, W. F., Aleksandrov, P., Qiu, X., and Lane, N. D. The future of large language model pre-training is federated. *arXiv preprint arXiv:2405.10853*, 2024. URL <https://arxiv.org/abs/2405.10853>.
- Sarfi, A., Thérien, B., Lidin, J., and Belilovsky, E. Communication efficient LLM pre-training with sparseloco. *CoRR*, abs/2508.15706, 2025. URL <https://doi.org/10.48550/arXiv.2508.15706>.
- Schaeffer, R., Levi, N., Kirsch, A., Guenais, T., Miranda, B., Obbad, E., and Koyejo, S. Evaluating the robustness of chinchilla compute-optimal scaling. *ICLR 2026 Conference Submission (OpenReview) and arXiv preprint arXiv:2509.23963*, 2025. URL <https://arxiv.org/pdf/2509.23963>.
- Shah, I., Polloreno, A. M., Stratos, K., Monk, P., Chaluvavaraju, A., Hojel, A., Ma, A., Thomas, A., Tanwer, A., Shah, D. J., Nguyen, K., Smith, K., Callahan, M., Pust, M., Parmar, M., Rushton, P., Mazarakis, P., Kapila, R., Srivastava, S., Singla, S., Romanski, T., Vanjani, Y., and Vaswani, A. Practical efficiency of muon for pre-training, 2025a. URL <https://arxiv.org/abs/2505.02222>.
- Shah, I., Polloreno, A. M., Stratos, K., Monk, P., Chaluvavaraju, A., Hojel, A., Ma, A., Thomas, A., Tanwer, A., Shah, D. J., Nguyen, K., Smith, K., Callahan, M., Pust, M., Parmar, M., Rushton, P., Mazarakis, P., Kapila, R., Srivastava, S., Singla, S., Romanski, T., Vanjani, Y., and Vaswani, A. Practical efficiency of muon for pre-training. *CoRR*, abs/2505.02222, 2025b. URL <https://doi.org/10.48550/arXiv.2505.02222>.
- Shallue, C. J., Lee, J., Antognini, J., Sohl-Dickstein, J., Frostig, R., and Dahl, G. E. Measuring the effects of data parallelism on neural network training. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- Shi, S., Chu, X., Cheung, K. C., and See, S. Understanding top-k sparsification in distributed deep learning. *CoRR*, abs/1911.08772, 2019.
- Stich, S. U. Local sgd converges fast and communicates little. *arXiv preprint arXiv:1805.09767*, 2018.
- Stich, S. U., Cordonnier, J., and Jaggi, M. Sparsified SGD with memory. *CoRR*, abs/1809.07599, 2018.
- Su, D., Kong, K., Lin, Y., Jennings, J., Norick, B., Kliegl, M., Patwary, M., Shoybi, M., and Catanzaro, B. Nemotron-cc: Transforming common crawl into a refined long-horizon pretraining dataset, 2024. URL <https://arxiv.org/abs/2412.02595>.
- Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., Chen, Z., Cui, J., Ding, H., Dong, M., Du, A., Du, C., Du, D., Du, Y., Fan, Y., Feng, Y., Fu, K., Gao, B., Gao, H., Gao, P., Gao, T., Gu, X., Guan, L., Guo, H., Guo, J., Hu, H., Hao, X., He, T., He, W., He, W., Hong, C., Hu, Y., Hu, Z., Huang, W., Huang, Z., Huang, Z., Jiang, T., Jiang, Z., Jin, X., Kang, Y., Lai, G., Li, C., Li, F., Li, H., Li, M., Li, W., Li, Y., Li, Y., Li, Z., Li, Z., Lin, H., Lin, X., Lin, Z., Liu, C., Liu, C., Liu, H., Liu, J., Liu, J., Liu, L., Liu, S., Liu, T. Y., Liu, T., Liu, W., Liu, Y., Liu, Y., Liu, Y., Liu, Y., Liu, Y., Liu, Z., Lu, E., Lu, L., Ma, S., Ma, X., Ma, Y., Mao, S., Mei, J., Men, X., Miao, Y., Pan, S., Peng, Y., Qin, R., Qu, B., Shang, Z., Shi, L., Shi, S., Song, F., Su, J., Su, Z., Sun, X., Sung, F., Tang, H., Tao, J., Teng, Q., Wang, C., Wang, D., Wang, F., Wang, H., Wang, J., Wang, J., Wang, J., Wang, S., Wang, S., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Z., Wang, Z., Wang, Z., Wei, C., Wei, Q., Wu, W., Wu, X., Wu, Y., Xiao, C., Xie, X., Xiong, W., Xu, B., Xu, J., Xu, J., Xu, L. H., Xu, L., Xu, S., Xu, W., Xu, X., Xu, Y., Xu, Z., Yan, J., Yan, Y., Yang, X., Yang, Y., Yang, Z., Yang, Z., Yang, Z., Yao, H., Yao, X., Ye, W., Ye, Z., Yin, B., Yu, L., Yuan, E., Yuan, H., Yuan, M., Zhan, H., Zhang, D., Zhang, H., Zhang, W., Zhang, X., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Z., Zhao, H., Zhao, Y., Zheng, H., Zheng, S., Zhou, J., Zhou, X., Zhou, Z., Zhu, Z., Zhuang, W., and Zu, X. Kimi k2: Open agentic intelligence, 2025. URL <https://arxiv.org/abs/2507.20534>.
- Villalobos, P., Ho, A., Sevilla, J., Besiroglu, T., Heim, L., and Hobbhahn, M. Will we run out of data? limits of llm scaling based on human-generated data, 2024.
- Vogels, T., Karimireddy, S. P., and Jaggi, M. Powersgd: Practical low-rank gradient compression for distributed optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL <https://arxiv.org/abs/1905.13727>.
- Vyas, N., Morwani, D., Zhao, R., Shapira, I., Brandfonbrener, D., Janson, L., and Kakade, S. M. SOAP: Improving and stabilizing shampoo using adam for language modeling. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=IDxZhXrpNf>.
- Wang, G., Qin, H., Jacobs, S. A., Holmes, C., Rajbhandari, S., Ruwase, O., Yan, F., Yang, L., and He, Y. Zero++: Extremely efficient collective communication for giant model training. *CoRR*, abs/2306.10209, 2023a. URL <https://doi.org/10.48550/arXiv.2306.10209>.
- Wang, J., Tantia, V., Ballas, N., and Rabbat, M. G. Slowmo:

- Improving communication-efficient distributed SGD with slow momentum. *CoRR*, abs/1910.00643, 2019.
- Wang, J., Lu, Y., Yuan, B., Chen, B., Liang, P., De Sa, C., Re, C., and Zhang, C. CocktailSGD: Fine-tuning foundation models over 500Mbps networks. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 36058–36076. PMLR, 23–29 Jul 2023b.
- Wang, X. and Aitchison, L. How to set adamw’s weight decay as you scale model and dataset size. *CoRR*, abs/2405.13698, 2024. URL <https://doi.org/10.48550/arXiv.2405.13698>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Zhang, G., Li, L., Nado, Z., Martens, J., Sachdeva, S., Dahl, G. E., Shallue, C. J., and Grosse, R. B. Which algorithmic choices matter at which batch sizes? insights from a noisy quadratic model. In Wallach, H. M., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E. B., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 8194–8205, 2019a. URL <https://proceedings.neurips.cc/paper/2019/hash/e0eacd983971634327ae1819ea8b6214-Abstract.html>.
- Zhang, H., Morwani, D., Vyas, N., Wu, J., Zou, D., Ghai, U., Foster, D. P., and Kakade, S. M. How does critical batch size scale in pre-training? In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.
- Zhang, M., Lucas, J., Ba, J., and Hinton, G. E. Lookahead optimizer: k steps forward, 1 step back. In Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019b. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/90fd4f88f588ae64038134f1eeaa023f-Paper.pdf.
- Zhao, J., Zhang, Z., Chen, B., Wang, Z., Anandkumar, A., and Tian, Y. Galore: Memory-efficient llm training by gradient low-rank projection, 2024. URL <https://arxiv.org/abs/2403.03507>.
- Zhou, L., He, Y., Zhai, K., Liu, X., Liu, S., Ma, X., Ye, G., Jiang, Y.-G., and Chai, H. Fedcada: Adaptive client-side optimization for accelerated and stable federated learning. *arXiv preprint arXiv:2405.11811*, 2024. URL <https://arxiv.org/abs/2405.11811>.

A. MuLoCo Algorithm Block

We present non-compressed and compressed versions of MuLoCo in Algorithms 1 and 2, respectively. The compressed version of the algorithm is used in the compression experiments of section 5, while Algorithm 1 is used elsewhere.

Algorithm 1 MuLoCo (No Compression).

```

1: Input:  $N$  (communication steps),  $K$  (workers),  $H$  (local
   steps),  $t$  (local step counter), data shards  $\{\mathcal{D}_1, \dots, \mathcal{D}_K\}$ ,
   initial model  $\theta^{(0)}$ , outer momentum  $\mu$ , outer LR  $\eta_{\text{out}}$ 
2:  $u^{(0)} \leftarrow 0$ ;  $\theta_i^{(1)} \leftarrow \theta^{(0)} \ \forall i$ ;  $t \leftarrow 0$ 
3: for  $n = 1 \dots N$  do
4:   for  $i = 1 \dots K$  in parallel do
5:     for  $h = 1 \dots H$  do
6:        $t \leftarrow t + 1$ ;  $(x, y) \sim \mathcal{D}_i$ ;  $\mathcal{L} \leftarrow f_\theta(x, y; \theta_i^{(t)})$ 
7:        $\theta_i^{(t)} \leftarrow \text{Muon}(\theta_i^{(t)}, \nabla \mathcal{L})$ 
8:     end for
9:      $\Delta_i^{(t)} \leftarrow \theta^{(t-H)} - \theta_i^{(t)}$  (parameter delta)
10:   end for
11:    $\Delta^{(t)} \leftarrow \frac{1}{K} \sum_{i=1}^K \Delta_i^{(t)}$  (all-reduce outer delta)
12:    $u^{(t)} \leftarrow \mu u^{(t-H)} + \eta_{\text{out}} \Delta^{(t)}$  (Outer Momentum)
13:    $\theta^{(t)} \leftarrow \theta^{(t-1)} - \mu u^{(t)} - \eta_{\text{out}} \Delta^{(t)}$  (Outer Update)
14: end for
    
```

Algorithm 2 MuLoCo with compressed communication and optional EF.

```

1: Input:  $N$  (communication steps),  $K$  (workers),  $H$  (local steps),
    $t$  (local step counter), data shards  $\{\mathcal{D}_1, \dots, \mathcal{D}_K\}$ , initial model
    $\theta^{(0)}$ , compressor  $\mathcal{C}$ , outer momentum  $\mu$ , outer LR  $\eta_{\text{out}}$ , error
   feedback flag EF
2:  $u^{(0)} \leftarrow 0$ ;  $\theta_i^{(1)} \leftarrow \theta^{(0)} \ \forall i$ ;  $t \leftarrow 0$ 
3: if EF then
4:    $\mathcal{E}_i^{(0)} \leftarrow 0 \ \forall i$ 
5: end if
6: for  $n = 1 \dots N$  do
7:   for  $i = 1 \dots K$  in parallel do
8:     for  $h = 1 \dots H$  do
9:        $t \leftarrow t + 1$ ;  $(x, y) \sim \mathcal{D}_i$ ;  $\mathcal{L} \leftarrow f_\theta(x, y; \theta_i^{(t)})$ 
10:       $\theta_i^{(t)} \leftarrow \text{Muon}(\theta_i^{(t)}, \nabla \mathcal{L})$ 
11:    end for
12:     $\Delta_i^{(t)} \leftarrow \theta^{(t-H)} - \theta_i^{(t)}$  (parameter delta)
13:    if EF then
14:       $\mathcal{E}_i^{(t)} \leftarrow \beta \mathcal{E}_i^{(t-H)} + \Delta_i^{(t)}$ 
15:       $\Delta_i^{(t)} \leftarrow \mathcal{C}(\mathcal{E}_i^{(t)})$  (compress EF)
16:       $\mathcal{E}_i^{(t+1)} \leftarrow \mathcal{E}_i^{(t)} - \Delta_i^{(t)}$ 
17:    else
18:       $\Delta_i^{(t)} \leftarrow \mathcal{C}(\Delta_i^{(t)})$  (compress parameter delta)
19:    end if
20:  end for
21:   $\Delta^{(t)} \leftarrow \frac{1}{K} \sum_{i=1}^K \Delta_i^{(t)}$  (all-reduce outer delta)
22:   $u^{(t)} \leftarrow \mu u^{(t-H)} + \eta_{\text{out}} \Delta^{(t)}$  (Outer Momentum)
23:   $\theta^{(t)} \leftarrow \theta^{(t-1)} - \mu u^{(t)} - \eta_{\text{out}} \Delta^{(t)}$  (Outer Update)
24: end for
    
```

B. Extended Background

We briefly present the necessary background for understanding our empirical evaluation of MuLoCo. This includes implementation details of Muon, a review of DiLoCo, a review of partitioned communication, a review of common compression algorithms for neural networks, and a review of efficient communication collectives used in tandem with these compression algorithms.

Muon is a recently proposed (Jordan et al., 2024) second-order optimizer that takes orthonormalized steps. Specifically, at each step, Muon updates its momentum accumulator,

$$m_t = \beta m_{t-1} + g_t,$$

and applies five steps of *Newton–Schulz* iteration (Björck & Bowie, 1971; Kovarik, 1970) to produce an orthogonalized update. Specifically, the iteration refines

$$X_{t,j} = p(X_{t,j-1} X_{t,j-1}^\top) X_{t,j-1},$$

where $p(x) = ax + bx^3 + cx^5$ with (a, b, c) empirically tuned to be 3.4445, −4.7750, 2.0315 (Jordan et al., 2024). This procedure effectively scales and rotates m_t toward the nearest orthogonal matrix, approximating $UV^\top$ when $m_t = U\Sigma V^\top$ is its singular value decomposition. The resulting orthogonalized update O_t is then rescaled by the learning rate and applied as $W_t = W_{t-1} - \eta O_t - \eta \lambda W_{t-1}$ with optional decoupled weight decay. In this work, we always use decoupled weight decay with Muon as it was shown to be important in maintaining Muon’s convergence benefits at larger model sizes (Liu et al., 2025).

DiLoCo (Douillard et al., 2023) is a distributed optimization algorithm that independently trains a single model with DP AdamW across K workers in parallel while only communicating across workers every H gradient descent steps. When individual workers span highly interconnected compute pools (e.g., single nodes, racks, adjacent racks, or entire datacenters for the largest models) with significantly faster bandwidth within pools than across them, DiLoCo can lead to significant speedups relative to DP training across the same number of accelerators. We present the original DiLoCo algorithm and highlight our modifications to it in Algorithm 1 and 2.

Partitioned Communication in Streaming DiLoCo. (Douillard et al., 2025) propose Streaming DiLoCo, a variant of DiLoCo which, among other modifications, reduces peak communication bandwidth by partitioning a model’s layers into J subsets. With a communication interval H (J must divide H), each subset j is communicated at an assigned multiple $j \cdot H/J$ with $1 \leq j \leq J$ of the communication interval. In this work we evaluate the performance of MuLoCo under partitioned communication.

Compressed Communication. Compressing neural network gradients and parameters has been well studied in the context of reducing communication costs (Rothchild et al., 2020; Stich et al., 2018; Wang et al., 2023b; Peng et al., 2024). This work studies two common compression algorithms: quantization and sparsification. We now briefly discuss how these compressors are applied to a single weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$:

- **Top- k sparsification** involves keeping the $k\%$ largest magnitude entries of $\mathbf{W}$. The remaining entries are set to 0 and are not communicated. Note that one must still communicate the sparsity pattern (Wang et al., 2023b).
- **Quantization** involves mapping the entries of $\mathbf{W}$ to a much smaller representable range, such that they can be efficiently encoded using an offset and a limited number of discrete levels from a codebook. The number of bits determines the size of the codebook, while the codebook itself defines the quantized values used to approximate entries in $\mathbf{W}$. In *linear* quantization, these levels are uniformly spaced over a fixed range, whereas *statistical* (or non-uniform) quantization allocates levels based on the empirical distribution of $\mathbf{W}$, typically assigning higher resolution to more frequently occurring values.

Collectives for compressed communication. A ring all-reduce is bandwidth-optimal in terms of communication volume, achieving the minimum possible per-worker data transfer for dense reductions (Wang et al., 2023a). However, exploiting compressed communication generally requires modifying the underlying collective. For example, Top- k sparsification necessitates an all-gather operation, whose bandwidth cost grows linearly with the number of workers.

For quantized communication, naively applying a ring-based reduce-scatter leads to error accumulation, since each hop requires a dequantize–reduce–quantize operation to preserve compression benefits. A natural remedy is to replace the ring reduce-scatter with an all-to-all reduce-scatter, which achieves the same aggregate bandwidth cost while performing reductions locally after receiving all peers’ quantized pseudogradients (Wang et al., 2023a).

Unlike prior work (Douillard et al., 2025; Basu et al., 2019), which abstracts away the choice of communication primitive, we explicitly model an efficient all-to-all reduce-scatter followed by a ring all-gather in our experiments. This design requires a single dequantize–reduce–quantize operation between the two collectives, resulting in two quantization and two dequantization operations per pseudogradient communication. To the best of our knowledge, this regime has not been empirically evaluated before.

Error feedback (EF) is a powerful technique for mitigating the bias introduced by lossy compression (Karimireddy et al., 2019). EF works by persisting the difference between the original update and its compressed version and incorporating it into future communications. Concretely, given a compressor $\mathcal{C}(\cdot)$ and a residual accumulator $\mathcal{E}_i$, worker i updates its EF accumulator $\mathcal{E}_i \leftarrow \beta \mathcal{E}_i + \Delta_i$, communicates the compressed update $\tilde{\Delta}_i = \mathcal{C}(\mathcal{E}_i)$, and removes the communicated value $\mathcal{E}_i \leftarrow \mathcal{E}_i - \tilde{\Delta}_i$, thereby only persisting what was not communicated. Error feedback has been shown to recover convergence guarantees comparable to uncompressed optimization for both sparsification and quantization under standard assumptions (Karimireddy et al., 2019). In our compression experiments, we ablate the effect of error feedback.

C. Related Work

We now contextualize our paper within the rich body of existing works on distributed and communication-efficient optimization (C.1,C.2), single-worker DiLoCo (C.3), and large batch training (C.4).

C.1. Distributed optimization with local steps

The idea of taking multiple local steps before communicating originated in the context of federated learning (McMahan et al., 2017), where the algorithm is known as Federated Averaging (FedAVG). In an i.i.d. setting, a variant of FedAVG known as Local SGD has been shown theoretically lead to communication savings (Stich, 2018). Models trained with LocalSGD were also found to have improved generalization relative to large batch training (Lin et al., 2018). Follow-up works have proposed more sophisticated server-side optimizers (Wang et al., 2019; Joseph et al., 2025; Reddi et al., 2021), while other research has focused on adaptive client-side optimizers (Zhou et al., 2024; Douillard et al., 2023; Sani et al., 2024). Notably, in the context of LLM pre-training, Douillard et al. (2023) propose DiLoCo, which uses SGD with Nesterov momentum as the server-side optimizer and AdamW on local workers. DiLoCo was demonstrated to scale to very large numbers of local steps ($H > 500$) (Douillard et al., 2023) and large models (Charles et al., 2025; Jaghouar et al., 2024b;a). Other recent works focus on studying DiLoCo variants that also communicate optimizer states, allowing them to prove convergence and improve performance at the cost of additional communication (Cheng & Glasgow, 2025; Iacob et al., 2026b;a; Jovanović et al., 2026). In a direct follow-up to the original DiLoCo work, Charles et al. (2025) study the performance of DiLoCo as model size and training data are scaled up. Crucially, the authors find that at each scale, DiLoCo’s performance degrades as the number of workers is increased, but that as the model scale is increased, the performance degradation decreases. We complement these findings in our work, by showing that MuLoCo improves on worker scaling over DiLoCo relative to their respective data-parallel baselines and that at large worker counts, MuLoCo also improves with scale.

C.2. Distributed optimizers with compressed communication

Beyond leveraging infrequent communication, many distributed optimizers directly reduce the number of bits communicated through compression. This is typically done by sparsifying (Stich et al., 2018; Shi et al., 2019; Peng et al., 2024), quantizing (Alistarh et al., 2017), sketching (Rothchild et al., 2020), computing low-rank approximations of the gradient (Zhao et al., 2024; Vogels et al., 2019; Ahn et al., 2025), or combining any number of these approaches (Wang et al., 2023b). Among these works, many leverage error feedback for mitigating the bias introduced by lossy compression (Karimireddy et al., 2019).

Some works combine local SGD or DiLoCo with various compression methods. In the context of image classification, Basu et al. (2019) apply top-k sparsification and quantization to error-corrected moving averages of parameter deltas. Streaming DiLoCo (Douillard et al., 2025) combines partitioned communication, overlapped communication, and quantization to reduce communication cost in DiLoCo. In concurrent work, SparseLoCo (Sarfi et al., 2025) demonstrates that using a local error feedback as an outer momentum allows reaching extreme compression ratios in DiLoCo without compromising performance.

C.3. Single worker DiLoCo

Charles et al. (2025) established that the performance of DiLoCo with only a single worker is consistently stronger than AdamW alone. This is partially a rediscovery of the lookahead optimizer (Zhang et al., 2019b) and reptile (Nichol et al., 2018; Abbes et al., 2025), except that single-worker DiLoCo uses a Nesterov-style update as opposed to vanilla SGD. In follow-up work, Kallusky et al. (2025) establishes that compared to lookahead, single-worker DiLoCo (or SNOO) performs substantially better. They also show that the performance of single-worker DiLoCo when training MoE and dense LLMs can lead to a substantial speedup over AdamW alone. In concurrent follow-up work, (Defazio et al., 2025) propose GPA, an algorithm that generalizes and improves single-worker DiLoCo by eliminating the outer optimizer and enabling smooth iterate averaging at every step. GPA is shown to outperform single-worker DiLoCo while requiring one less accumulator. In this work, we establish the performance of single-worker MuLoCo at $H=30$ inner steps, showing that it outperforms data parallel Muon and has larger critical batch sizes. To the best of our knowledge, we are the first to establish $K=1$ MuLoCo’s larger critical batch sizes and show that it obtains a better performance-training-time tradeoff than DP Muon. We expect that the performance improvements we establish will directly apply to GPA with a Muon inner optimizer.

C.4. Large-Batch Training and Critical Batch Size

Scaling batch size may be the simplest way to reduce training time, whether it be from increased data parallelism or improved hardware utilization. However, increasing batch size beyond a certain threshold, known as the critical batch size (CBS), leads to significant diminishing returns in performance. As such, many works have studied the large-batch training regime and its performance differences to using smaller batch sizes. Early empirical work highlighted that large batches can alter

optimization dynamics and lead to poorer generalization behaviour (Keskar et al., 2017; Jastrzebski et al., 2017). At the same time, practical recipes such as linear learning-rate scaling and warmup demonstrated that large-batch training can match small-batch accuracy when carefully tuned (Goyal et al., 2017). Other work found that the generalization gaps found in earlier work stemmed from insufficient hyperparameter tuning, rather than inherent limitations of batch size (Shallue et al., 2019). In the context of language modeling, McCandlish et al. (2018) introduced the *gradient noise scale* as a predictor of the CBS, hypothesizing that the CBS coincides with the point at which increasing batch size no longer meaningfully reduces noise in the gradient. Recent work has revisited and refined this perspective. Zhang et al. (2025) demonstrate that critical batch size scales primarily with dataset size rather than model size. Merrill et al. (2025) propose a direct empirical measurement of the critical batch size, while Bergsma et al. (2025) derive scaling laws relating optimal and critical batch sizes to dataset size, providing an alternative methodology for predicting the CBS. Finally, several works establish that the critical batch size depends on the optimization algorithm (Zhang et al., 2019a; Lin et al., 2020). For instance, in the LLM regime, Vyas et al. (2025) show empirically that second-order preconditioned methods such as SOAP tolerate larger batches than AdamW, while Shah et al. (2025b) demonstrate that Muon can also reach larger critical batch sizes. Our study further expands on this, showing that MuLoCo achieves a Pareto optimal performance-batch size trade-off among all optimizers in our study (notably including DP Muon).

D. Extended Pseudogradient analysis & Proof of proposition 3.1

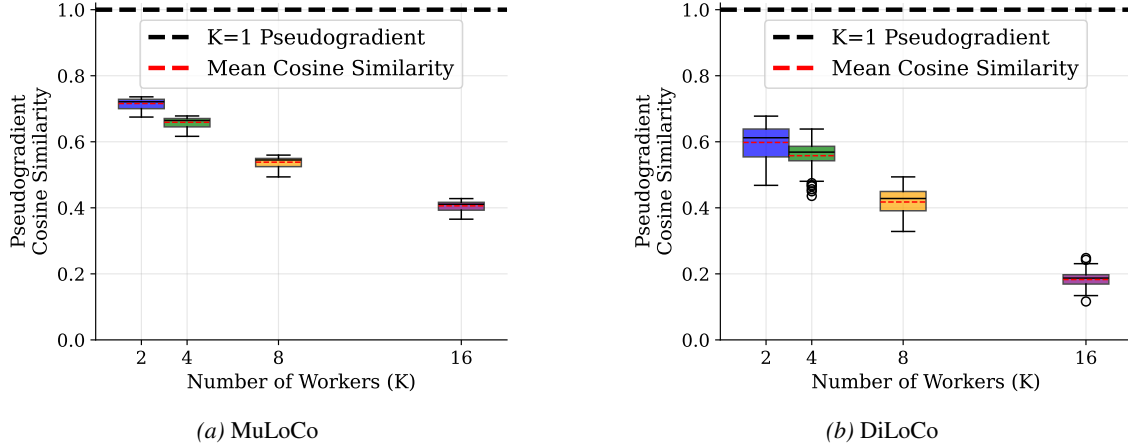


Figure 8. **Pseudogradient cosine similarities compared to a FLOP-matched $K = 1$ pseudogradient.** Subfigure (a) shows results for Muon, while subfigure (b) shows results for AdamW. The box plots report the spread of cosine similarities across all hidden weight matrices of a 416M transformer. We observe that Muon’s pseudogradients are more aligned with the $K=1$ pseudogradient, and this alignment decreases more slowly as the number of workers is increased.

D.1. Analyzing the benefits of Muon pseudogradients over AdamW

Our results (Sec. 5) will show that beyond improving over DiLoCo in absolute terms, MuLoCo improves performance at larger worker counts over DiLoCo even when normalized by its own data-parallel baseline. To better understand the reasons for this improvement we ran a number of analytical experiments targeted towards analyzing the impact of Muon and AdamW on pseudogradient quality. In the remainder of this section we will summarize the important results as they relate to our theoretical analysis.

Figure 8 reports the average cosine similarity across all hidden layers in a 12-layer transformer (416M in Table 1) for DiLoCo and MuLoCo at $K \in \{2, 4, 8, 16\}$ workers. The cosine similarity is computed between vectorized tensors and their corresponding $K=1$ pseudogradients. We find that at each worker count, MuLoCo’s pseudogradient is more aligned with the $K=1$ pseudogradient (equivalent to the data-parallel weight difference). Moreover, the variance in cosine similarity across hidden tensors in the network is much larger for AdamW than Muon. These findings suggest that improved alignment may be a key factor in MuLoCo’s improved performance at larger worker counts

In an attempt to better understand the exacerbated pseudogradient degradation at higher worker counts for DiLoCo relative to MuLoCo, in figure 9 (a) we visualize the pseudogradient spectra of DiLoCo and MuLoCo before and after averaging. We observe that the spectrum of the individual worker’s pseudogradients collapse for AdamW during averaging, while the collapse is much less pronounced for muon. To measure this we introduce the interference gap.

Definition D.1 (top- S interference gap). Let $A_1, \dots, A_N \in \mathbb{R}^{m \times n}$ and define their mean

$$\bar{A} = \frac{1}{N} \sum_{i=1}^N A_i.$$

Denote by $\sigma_1(X) \geq \dots \geq \sigma_r(X) \geq 0$ the singular values of a matrix $X \in \mathbb{R}^{m \times n}$ with $r = \min\{m, n\}$. For any $k \in \{1, \dots, r\}$, the top- S interference gap is

$$G_S = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^S \sigma_j(A_i) - \sum_{j=1}^S \sigma_j(\bar{A}).$$

Here, $G_S \geq 0$ measures the loss of top- S spectral mass when averaging the matrices: $G_S = 0$ indicates perfect alignment of dominant singular directions, while larger G_S implies stronger destructive interference.

Figure 9 (b) reports the top 5% interference gap observed for MuLoCo and DiLoCo at different worker counts, finding that G_S decreases with worker count for MuLoCo but not DiLoCo. These findings reveal differences in behaviour between

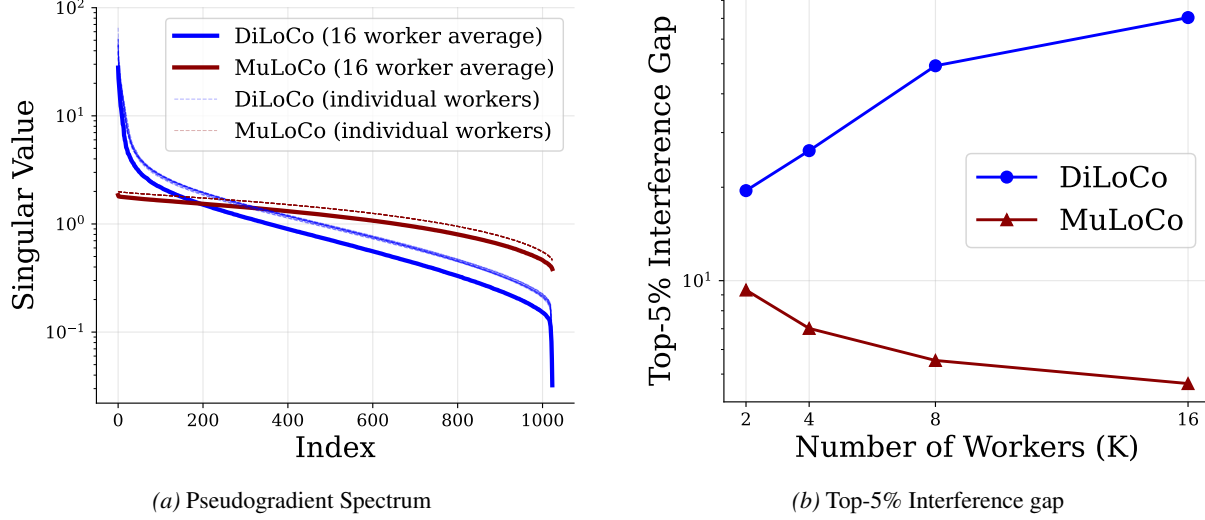


Figure 9. **DiLoCo’s pseudogradient spectrum decays relative to the spectrum of its trajectories as the number of workers increases.** (a) We plot the singular values of each worker weight difference Δ_k (dashed) and of the averaged pseudogradient Ψ (solid), where $\Psi = \frac{1}{K} \sum_{k=1}^K \Delta_k$. (b) We quantify destructive interference via the *top-S interference gap* $G_k = \frac{1}{K} \sum_{k=1}^K \sum_{j=1}^S \sigma_j(\Delta_k) - \sum_{j=1}^S \sigma_j(\Psi)$, i.e., the difference between the mean top-S Ky–Fan spectral mass of individual worker updates and the top-S spectral mass of their average. We observe that DiLoCo’s interference gap grows as the number of workers increases, while Muon’s shrinks, providing a possible explanation for DiLoCo’s stronger performance degradation at large worker counts.

AdamW and Muon pseudogradients, suggesting that top-S interference may be related to the observed performance improvements for MuLoCo.

D.2. Relating Muon and AdamW Pseudogradient Spectra to Per-step Alignment

Inspired by our empirical results of the previous section, which show a meaningful difference between DiLoCo and MuLoCo pseudogradients in terms of alignment with the $K=1$ pseudogradient and top-S interference during averaging, in this section, we theoretically link the two. Specifically, we show that the nuclear norm of MuLoCo and DiLoCo pseudogradients is directly related to their alignment with individual optimizer steps.

AdamW inner updates and induced pseudogradient. For a given worker k and local step $h \in \{1, \dots, H\}$ with gradient $g_k^{(t+h)}$, AdamW maintains first and second moments

$$\begin{aligned} m_{k,h} &= \beta_1 m_{k,h-1} + (1 - \beta_1) g_k^{(t+h)}, \\ v_{k,h} &= \beta_2 v_{k,h-1} + (1 - \beta_2) g_k^{(t+h)} \odot g_k^{(t+h)}, \end{aligned}$$

and applies decoupled weight-decay leading to the following worker trajectory over H local steps

$$\Delta_{\text{AdamW}}^{(t,k)} = \sum_{h=1}^H \left(\alpha_{t+h} \frac{m_{k,h}}{\sqrt{v_{k,h}} + \varepsilon} + \lambda \alpha_{t+h} \theta_k^{(t+h-1)} \right).$$

Muon inner updates and induced pseudogradient. With decoupled weight decay, Muon’s orthonormalized update yield the following worker trajectory:

$$\Delta_{\text{Muon}}^{(t,k)} = \sum_{h=1}^H \left(\alpha_{t+h} O_{k,h} + \lambda \alpha_{t+h} \theta_k^{(t+h-1)} \right).$$

For a more detailed explanation of Muon’s update, we refer the reader to section 2.

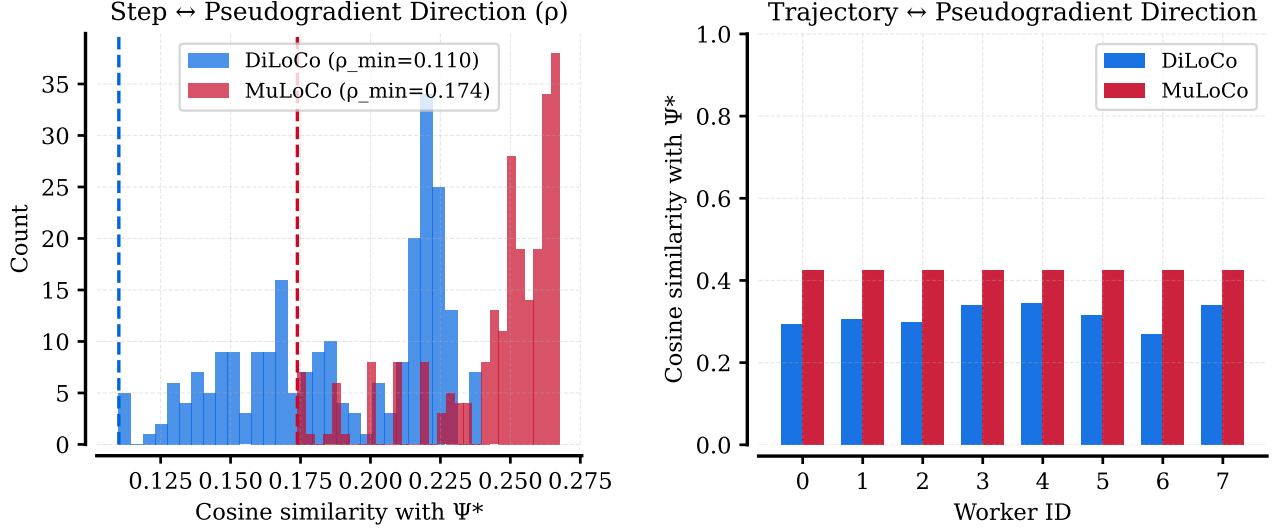


Figure 10. **Alignment to the full pseudogradient.** We report (left) the cosine similarity between *individual inner-optimizer steps* and the full communicated pseudogradient, and (right) the cosine similarity between *per-worker weight differences* (i.e., worker trajectories) and the full pseudogradient. We observe that MuLoCo’s (Muon inner) optimizer steps are more consistently aligned with the final pseudogradient direction. Moreover, worker weight differences under Muon exhibit nearly identical cosine similarity to the final pseudogradient across workers, while AdamW shows considerably higher inter-worker variability.

Since the weight decay hyperparameter for Muon and AdamW is generally chosen to be $\lambda \leq 10^{-1}$ (this is the case in all our experiments), the contribution of decoupled weight decay to the pseudogradient is negligible. As such, we exclude it from our analysis. We now demonstrate that the nuclear norm of general pseudogradients depends on the alignment of individual optimizer steps with the pseudogradient and on the Frobenius norm of the optimizer step, itself.

Notation. For a matrix $A \in \mathbb{R}^{m \times n}$, we denote by $\sigma_1(A) \geq \dots \geq \sigma_r(A) \geq 0$ its singular values, with $r := \min\{m, n\}$. The Frobenius inner product is $\langle A, B \rangle_F := \text{Tr}(A^\top B)$, and the Frobenius norm is $\|A\|_F := \sqrt{\langle A, A \rangle_F}$. The nuclear norm is $\|A\|_* := \sum_{j=1}^r \sigma_j(A)$.

Proposition D.2 (Pseudogradient nuclear norm depends on optimizer step alignment). *Fix $m, n \in \mathbb{N}$ and let $r := \min\{m, n\}$. Consider a pseudogradient of the form*

$$\Psi = \frac{1}{K} \sum_{k=1}^K \sum_{h=1}^H \alpha_{t+h} \psi^{(t+h,k)} \in \mathbb{R}^{m \times n},$$

where $\alpha_{t+h} \geq 0$ is the learning rate and $\psi^{(t+h,k)}$ is the optimizer step. Let $\Psi = U\Sigma V^\top$ be its singular value decomposition and define the orthonormal factor $\Psi^* := UV^\top$. Let the cosine similarity between a given optimizer step and Ψ^* be

$$\rho^{(t+h,k)} := \frac{\langle \psi^{(t+h,k)}, \Psi^* \rangle_F}{\|\psi^{(t+h,k)}\|_F \|\Psi^*\|_F}.$$

Then the nuclear norm of Ψ satisfies

$$\|\Psi\|_* = \frac{\sqrt{r}}{K} \sum_{k=1}^K \sum_{h=1}^H \rho^{(t+h,k)} \alpha_{t+h} \|\psi^{(t+h,k)}\|_F.$$

Proof. Let $\Psi = U\Sigma V^\top$ be the singular value decomposition of Ψ , and recall the definition of its *orthonormal factor* $\Psi^* := UV^\top$.

We first show that $\langle \Psi, \Psi^* \rangle_F = \|\Psi\|_*$. Indeed,

$$\begin{aligned} \langle \Psi, \Psi^* \rangle_F &= \text{Tr}(\Psi^\top \Psi^*) = \text{Tr}((U\Sigma V^\top)^\top (UV^\top)) \\ &= \text{Tr}((V\Sigma U^\top)(UV^\top)) = \text{Tr}(V\Sigma(U^\top U)V^\top) \\ &= \text{Tr}(V\Sigma V^\top) = \text{Tr}(\Sigma) = \sum_{j=1}^r \sigma_j(\Psi) = \|\Psi\|_*. \end{aligned} \quad (5)$$

By linearity of the Frobenius inner product, we can expand

$$\begin{aligned} \langle \Psi, \Psi^* \rangle_F &= \left\langle \frac{1}{K} \sum_{k=1}^K \sum_{h=1}^H \alpha_{t+h} \psi^{(t+h,k)}, \Psi^* \right\rangle_F \\ &= \frac{1}{K} \sum_{k=1}^K \sum_{h=1}^H \alpha_{t+h} \langle \psi^{(t+h,k)}, \Psi^* \rangle_F. \end{aligned} \quad (6)$$

Substituting the definition of cosine similarity,

$$\rho^{(t+h,k)} := \frac{\langle \psi^{(t+h,k)}, \Psi^* \rangle_F}{\|\psi^{(t+h,k)}\|_F \|\Psi^*\|_F},$$

we obtain

$$\langle \Psi, \Psi^* \rangle_F = \frac{1}{K} \sum_{k=1}^K \sum_{h=1}^H \rho^{(t+h,k)} \alpha_{t+h} \|\psi^{(t+h,k)}\|_F \|\Psi^*\|_F. \quad (7)$$

Finally, since $\Psi^* = UV^\top$ has singular values equal to 1 for $j \leq r$, we have

$$\|\Psi^*\|_F = \sqrt{\sum_{j=1}^r \sigma_j(\Psi^*)^2} = \sqrt{r}.$$

Combining this with equation 5, equation 7 yields

$$\|\Psi\|_* = \langle \Psi, \Psi^* \rangle_F = \frac{\sqrt{r}}{K} \sum_{k=1}^K \sum_{h=1}^H \rho^{(t+h,k)} \alpha_{t+h} \|\psi^{(t+h,k)}\|_F,$$

□

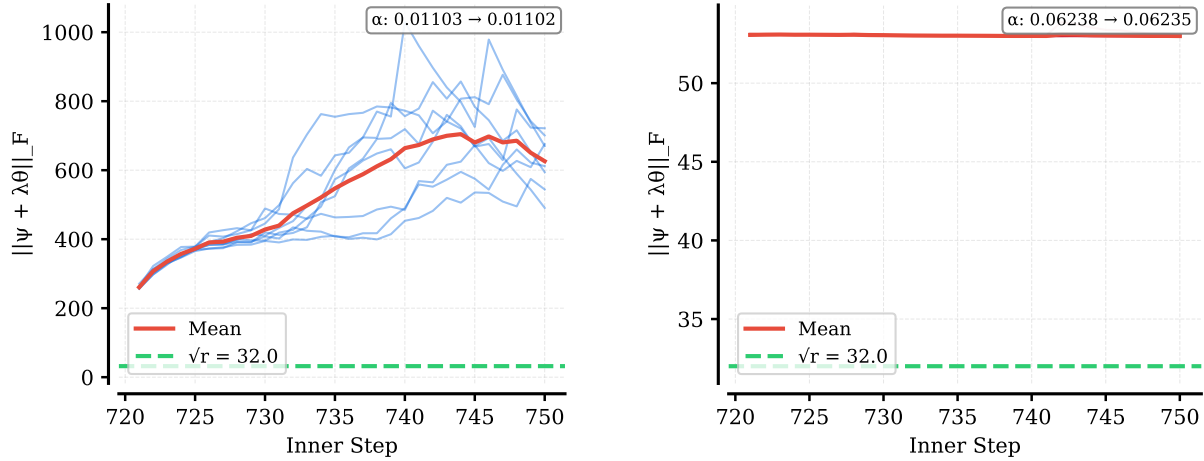
Corollary D.3 (Muon pseudogradient). *In the setting of Proposition 3.1, assume that the inner-step optimizer updates, $\psi^{(t+h,k)}$, are produced by Newton Shulz Iteration, making them orthonormal; then*

$$\|\Psi\|_* = \frac{r}{K} \sum_{k=1}^K \sum_{h=1}^H \rho^{(t+h,k)} \alpha_{t+h}.$$

Proof. This follows from proposition 3.1 since $\psi^{(t+h,k)}$ is orthonormal we have $\|\psi^{(t+h,k)}\|_F = \sqrt{r}$. □

Corollary D.4 (AdamW pseudogradient). *In the setting of Proposition 3.1, for general inner-step matrices $\psi^{(t+h,k)}$ (e.g., AdamW effective steps), one always has*

$$\|\Psi\|_* = \frac{\sqrt{r}}{K} \sum_{k=1}^K \sum_{h=1}^H \rho^{(t+h,k)} \alpha_{t+h} \|\psi^{(t+h,k)}\|_F.$$


 (a) Frobenius norm of **DiLoCo** inner steps.

 (b) Frobenius norm of **MuLoCo** inner steps.

Figure 11. MuLoCo’s orthonormalized inner steps lead to stable Frobenius norms. Light blue (DiLoCo) and light red (MuLoCo) lines correspond to the Frobenius norm of the inner-optimizer step on each individual worker, while the red line reports their mean (MuLoCo’s mean overlaps the individual workers). We observe that AdamW’s per-worker step norms are comparatively erratic across workers, whereas Muon’s are stable. Muon’s step Frobenius norms are larger than the predicted $\sqrt{r}$ magnitude (for rank r) due to the additional contribution of decoupled weight decay.

Proof. This is a restatement of proposition 3.1 to illustrate that it applies to AdamW pseudogradients. $\square$

We observe that Muon’s orthogonalized optimizer step results in the pseudogradient’s nuclear norm naturally varying with the per-step alignment (ρ), the number of workers (K), and the inner learning rate (α). In contrast, the spectrum of AdamW’s pseudogradient also depends on $\|\psi^{(t+h,k)}\|_F$, which may be highly variable at higher worker counts, leading to overemphasizing certain steps unnecessarily. This exact behaviour is illustrated in Figure 11 where we empirically measure the Frobenius norm of individual optimizer steps, forming the pseudogradient for DiLoCo and MuLoCo. Subfigure (a) shows DiLoCo’s pseudogradient Frobenius norm is highly variable across different workers at each inner step, while subfigure (b) shows that MuLoCo’s is constant throughout. Intuitively, Muon’s orthonormalized optimizer steps allow each individual optimizer step to contribute equally to the pseudogradient’s spectrum. While it should be noted that our theory has no direct connection to performance and that all inferences made here are purely speculative, the slower decrease in cosine similarity of MuLoCo in Figure 8 relative to DiLoCo is correlated with its relatively stronger performance at larger worker counts (Figure 1 (a)). Together with our theory connecting the pseudogradients nuclear norm to the cosine similarity and Frobenius norm of individual optimizer steps, we speculate that the stronger alignment of MuLoCo’s optimizer steps and its stable Frobenius norm across them may lead to a larger pseudogradient spectrum with less interference during averaging (e.g., as observed in Figure 9), which may improve performance.

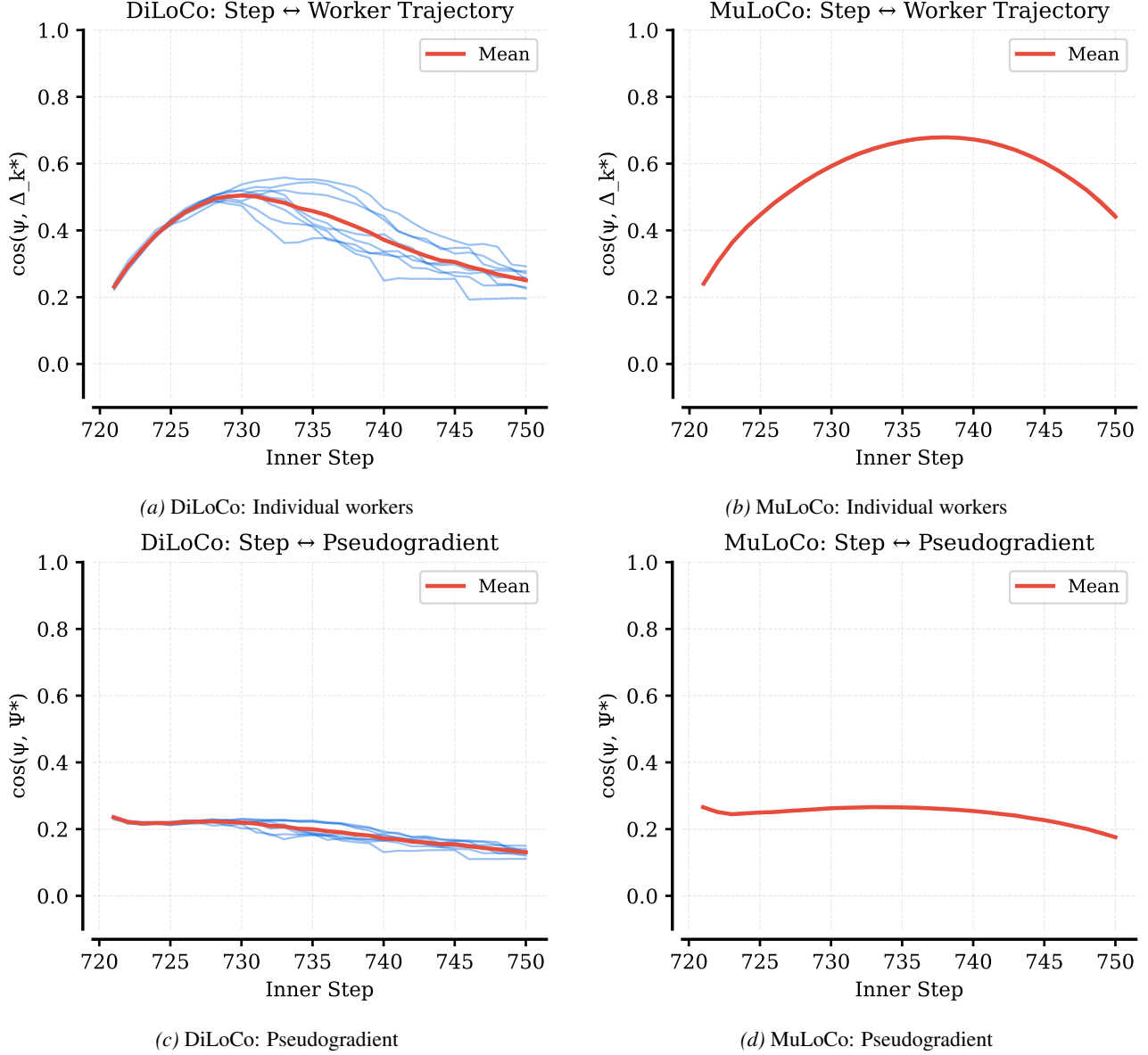


Figure 12. **Worker-level variability in step alignment for DiLoCo vs. MuLoCo.** Each plot overlays the trajectories of *individual workers* together with their *mean* (labeled “Mean”). For MuLoCo (right column), the mean curve lies almost exactly on top of the individual-worker curves, making them largely indistinguishable. In contrast, DiLoCo (left column) exhibits visibly separated worker trajectories, indicating substantially higher variance across workers. Top row shows individual-worker trajectories; bottom row shows the corresponding pseudogradient views.

E. Extended Results

This section reviews complementary and additional results to the main manuscript: (E.1) additional experiments regarding compressed communication, (E.3) additional scaling law results, and (E.4) additional results detailing our 15B training runs.

E.1. MuLoCo vs DiLoCo: Pseudogradient Compression

Figure 13 reports the performance for all quantization methods, including row-wise versions of statistical and linear quantization. We observe that across all settings, 4-bit quantization is effectively lossless and 2-bit quantization remains strong, *except* for global linear quantization, which shows a clear degradation at 2 bits. In all cases, MuLoCo outperforms DiLoCo, and enabling EF provides little additional benefit. In figure 15 we observe that even quantizing to 4-bits can have a substantial advantage when communication is the bottleneck. As demonstrated in our experiments, this comes at no cost to performance.

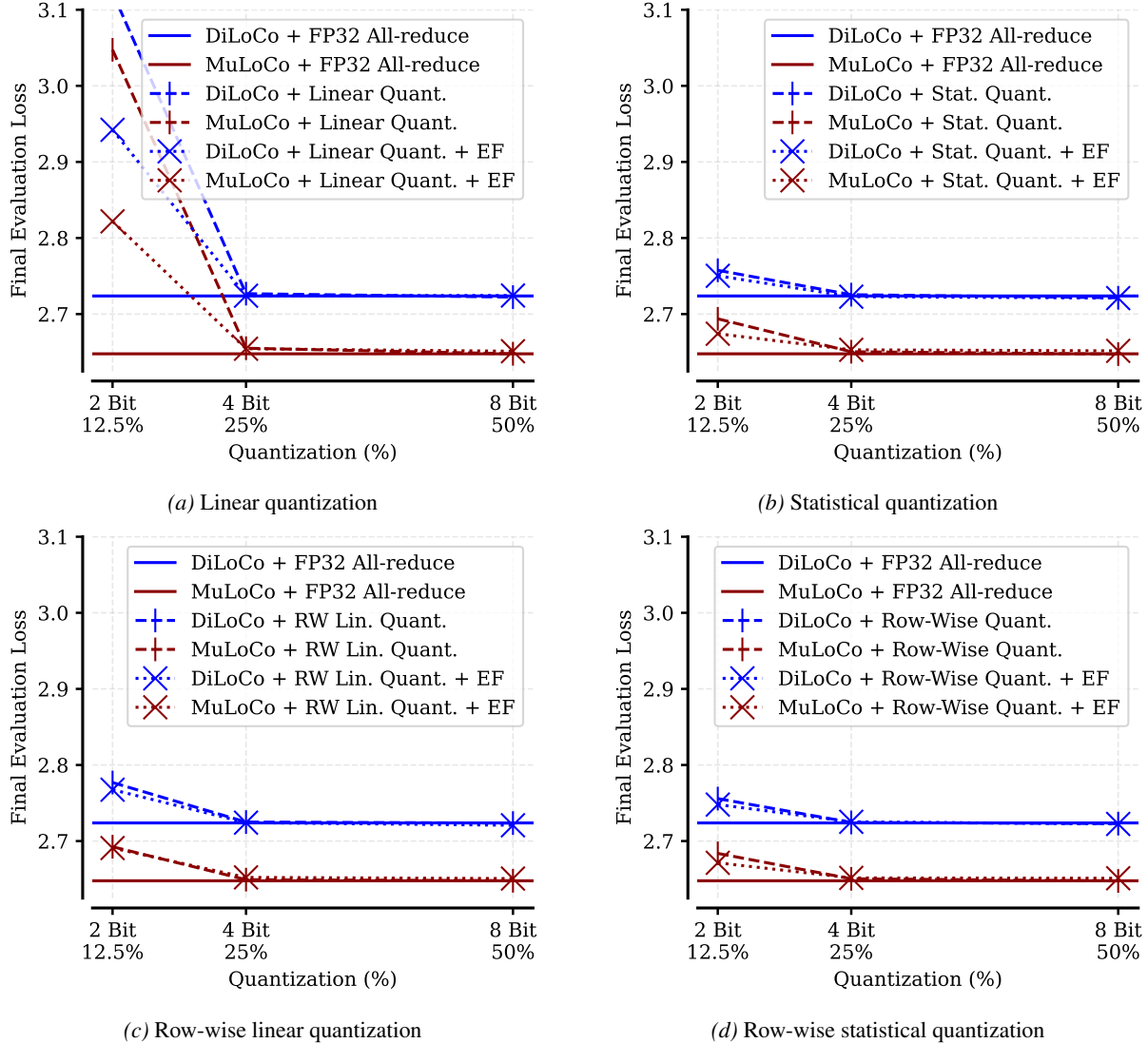


Figure 13. **Quantization ablations.** We compare MuLoCo and DiLoCo under 8-bit (50%), 4-bit (25%), and 2-bit (12.5%) quantized communication, with and without error feedback (EF). Across all settings, 4-bit quantization is effectively lossless and 2-bit quantization remains strong, *except* for global linear quantization, which shows a clear degradation at 2 bits. In all cases, MuLoCo outperforms DiLoCo, and enabling EF provides little additional benefit.

Global Quantization v.s. Row-wise quantization. With simple and efficient implementations in mind, we decided to study

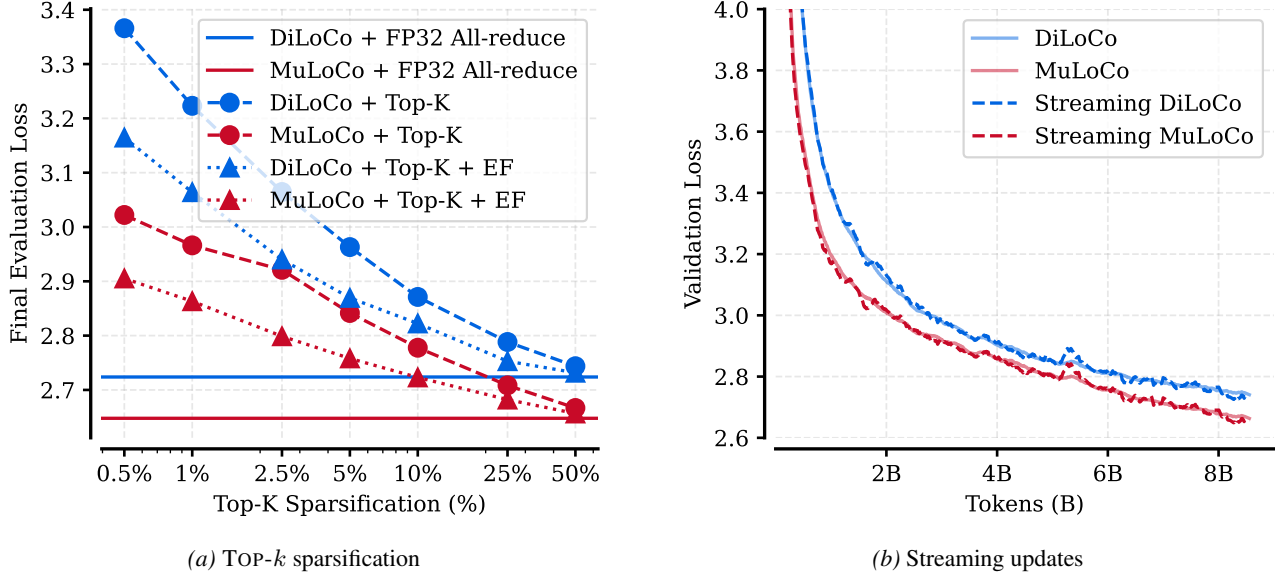


Figure 14. **Top-k compression and streaming.** **Left (Top-k):** MuLoCo remains more robust than DiLoCo at higher sparsities (i.e., smaller communicated fractions), achieving lower final validation loss at aggressive sparsification levels. Error feedback (EF) improves both methods, but does not make extreme sparsities viable: performance degrades rapidly as sparsity increases. **Right (Streaming):** Streaming and non-streaming variants match closely for both DiLoCo and MuLoCo, indicating no degradation from partitioned (streamed) communication. In particular, MuLoCo tracks DiLoCo under streaming and remains fully compatible with streaming updates.

both global and row-wise quantization. Global quantization is simple to implement and has minimal metadata overhead; however, it does require computing statistics across the entire tensor, which can be inefficient for large models or when parameters are sharded. Row-wise quantization was chosen for its improved performance characteristics and its affinity for parallelization. While the row-wise approach has the requirement to store metadata for each row, this avoids the need to synchronize metadata and tensor value information during the reduction since everything can be sent at once. It is also amenable to highly parallelized dequantize-reduce-quantize operations on GPUs since each row can be processed independently.

Assumptions around the collective used. For our quantization experiments, we assume that collective communication is implemented using a bandwidth-efficient all-to-all reduce-scatter and subsequent ring all-gather (Wang et al., 2023a). This choice is made instead of (1) using a bandwidth-expensive all-gather of full quantized parameters and (2) using a ring for the reduce-scatter. Compared to (1), the all-to-all allows us to avoid bandwidth growth in the number of workers, while relative to (2) it avoids compounding quantization errors during the ring-reduction, which is also exacerbated as the number of workers is increased. When using an all-to-all collective for the reduce scatter, all quantized tensors can be reduced in high precision on the same device before being re-quantized for the final all-gather. This required two total quantizations: (1) a quantization before the all-to-all, and (2) a quantization after the high-precision reduction before the all-gather. All our quantization experiments use two quantizations. For our top-k experiments, we assume an all-gather across all parameters and, therefore, only sparsify the tensor once immediately before communication.

Compression results in table format For the reader’s convenience, we provide our quantization and top-k results in Tables 4 and 3, respectively.

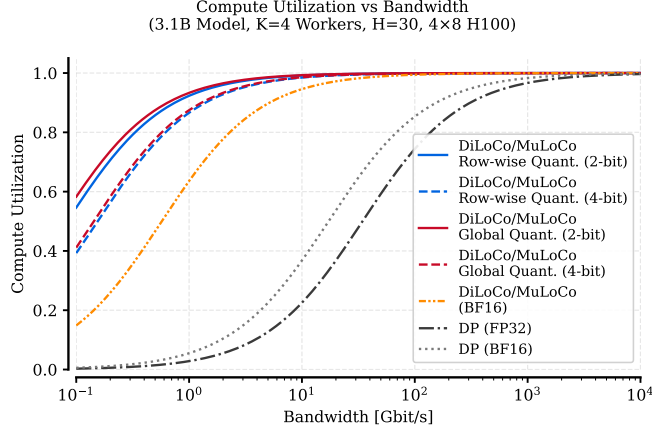


Figure 15. **Compute Utilization as a function of network bandwidth when training a 3.1B model with different optimizers and compression schemes.** The plot was using timings from Table 4 and assumes that any time not spent communicating is used for computation. We observe that DiLoCo and MuLoCo baselines require two orders of magnitude less bandwidth to achieve 99%+ compute utilization than DP approaches.

Table 3. Final evaluation loss for top- k sparsification.

Top-K	DiLoCo		MuLoCo	
	w/o EF	w/ EF	w/o EF	w/ EF
FP32 (baseline)	2.7238		2.6478	
0.5%	3.3660	3.1646	3.0221	2.9052
1%	3.2231	3.0641	2.9662	2.8633
2.5%	3.0644	2.9402	2.9211	2.7988
5%	2.9630	2.8695	2.8417	2.7580
10%	2.8712	2.8224	2.7777	2.7230
25%	2.7883	2.7529	2.7084	2.6822
50%	2.7437	2.7316	2.6668	2.6566

Table 4. Final evaluation loss for different quantization strategies.

Compressor	Bits	DiLoCo		MuLoCo	
		w/o EF	w/ EF	w/o EF	w/ EF
FP32 (baseline)	-	2.7238		2.6478	
Linear	2-bit	2.9819	2.8695	2.7751	2.7543
	4-bit	2.7268	2.7241	2.6551	2.6544
	8-bit	2.7223	2.7244	2.6478	2.6512
Statistical	2-bit	2.7576	2.7503	2.6938	2.6742
	4-bit	2.7255	2.7229	2.6505	2.6531
	8-bit	2.7214	2.7210	2.6474	2.6517
RW Linear	2-bit	2.7769	2.7675	2.6925	2.6908
	4-bit	2.7250	2.7242	2.6492	2.6522
	8-bit	2.7236	2.7208	2.6478	2.6507
RW Statistical	2-bit	2.7558	2.7480	2.6838	2.6714
	4-bit	2.7245	2.7253	2.6504	2.6516
	8-bit	2.7229	2.7225	2.6476	2.6511

E.2. Scaling Laws Fit Hyperparameters

We fit the parameters of all scaling laws by minimizing a Huber loss (Huber, 1964) in log-space. Specifically, given predictions $\hat{L}_i$ and targets L_i , we minimize $\sum_i H_\delta(\log \hat{L}_i - \log L_i)$ where H_δ is the Huber loss function with $\delta = 0.001$. This provides robustness to outliers while remaining smooth near zero. Optimization is performed using L-BFGS (Nocedal, 1980) with a maximum of 15,000 iterations per run. To avoid poor local minima, each fit is repeated from 512 random initializations, and the solution with the lowest objective is retained. For joint irreducible loss fitting, a three-phase grid search is used: a coarse sweep over 200 candidate L_{irr} values (each evaluated with 64 restarts per combination), a zoom phase refining the neighborhood of the best candidate, and a final phase re-fitting all combinations at the selected L_{irr} with the full 512 restarts.

E.3. Extended Scaling Law Experiments

The following section presents further analysis and reports on greater details of our scaling law experiments. Table 5 reports the parameters of scaling laws fit in section 6. We also report the final loss values of all methods in Table 6 and the zero-shot evaluation benchmark scores for 3.1B models in Table 7.

Table 5. **Parameters for $L(C) = aC^\alpha + L_0$** with joint $L_0 = 1.711$, fitted on all five training scales (150M–3.1B). Residuals are $|\log L_{\text{actual}} - \log L_{\text{pred}}|$.

Method	K	a	α	Train	15B
DP AdamW	1	5677	−0.195	0.003	0.025
DP Muon	1	6584	−0.199	0.002	0.032
DiLoCo	1	6620	−0.199	0.004	0.032
	2	5647	−0.195	0.003	—
	4	5640	−0.194	0.002	—
	8	6015	−0.195	0.002	—
	16	5372	−0.191	0.001	0.040
MuLoCo	1	6927	−0.200	0.002	0.028
	2	6852	−0.200	0.003	—
	4	6851	−0.200	0.002	—
	8	6467	−0.198	0.004	—
	16	6906	−0.199	0.002	0.026
Mean				0.002	0.030

Table 6. **MuLoCo vs DiLoCo scaling study.** Percentages show relative change from respective DP baselines. Bold = best (lowest) loss in each column.

	Method	150M	416M	914M	1.8B	3.1B	15B (No Sweep)
DP	Muon	3.124	2.641	2.402	2.246	2.128	1.875
	AdamW	3.158	2.682	2.440	2.266	2.145	1.901
$K=1$	MuLoCo	3.120 (−0.1%)	2.638 (−0.1%)	2.400 (−0.1%)	2.238 (−0.4%)	2.122 (−0.3%)	1.878 (+0.2%)
	DiLoCo	3.142 (−0.5%)	2.650 (−1.2%)	2.411 (−1.2%)	2.265 (−0.1%)	2.136 (−0.4%)	1.877 (−1.3%)
$K=2$	MuLoCo	3.130 (+0.2%)	2.642 (+0.0%)	2.402 (+0.0%)	2.245 (+0.0%)	2.127 (−0.1%)	—
	DiLoCo	3.159 (+0.0%)	2.668 (−0.5%)	2.428 (−0.5%)	2.275 (+0.4%)	2.155 (+0.5%)	—
$K=4$	MuLoCo	3.148 (+0.8%)	2.651 (+0.4%)	2.410 (+0.4%)	2.254 (+0.4%)	2.139 (+0.5%)	—
	DiLoCo	3.192 (+1.1%)	2.697 (+0.6%)	2.447 (+0.3%)	2.283 (+0.7%)	2.166 (+1.0%)	—
$K=8$	MuLoCo	3.177 (+1.7%)	2.659 (+0.7%)	2.426 (+1.0%)	2.273 (+1.2%)	2.155 (+1.3%)	—
	DiLoCo	3.248 (+2.8%)	2.734 (+1.9%)	2.478 (+1.6%)	2.300 (+1.5%)	2.181 (+1.7%)	—
$K=16$	MuLoCo	3.222 (+3.1%)	2.713 (+2.8%)	2.448 (+1.9%)	2.291 (+2.0%)	2.165 (+1.7%)	1.902 (+1.4%)
	DiLoCo	3.326 (+5.3%)	2.808 (+4.7%)	2.522 (+3.4%)	2.348 (+3.6%)	2.215 (+3.3%)	1.910 (+0.4%)

Table 7. **Zero-shot LM Evaluation Results all optimizers at 3.1B scale and DP, K=1, and K=16 at 15B scale.** We report accuracy for MMLU, WinoGrande and normalized accuracy for HellaSwag, PIQA, ARC-E, ARC-C, OBQA. We additionally report mean evaluation accuracy across all available tasks.

Model Size	Configuration	MMLU	HellaSwag	WinoGrande	PIQA	ARC-E	ARC-C	OBQA	Mean Acc.
3.1B	DP AdamW	29.76%	66.98%	60.62%	76.33%	71.17%	42.66%	41.40%	55.56%
3.1B	DP Muon	37.24%	68.18%	62.95%	77.49%	71.19%	42.03%	42.47%	57.36%
3.1B	DiLoCo K=1	35.34%	67.58%	64.09%	77.58%	69.36%	41.81%	41.80%	56.79%
3.1B	DiLoCo K=2	27.46%	66.48%	61.40%	76.66%	69.23%	41.30%	40.80%	54.76%
3.1B	DiLoCo K=4	30.01%	65.84%	61.64%	76.82%	70.66%	41.30%	41.80%	55.44%
3.1B	DiLoCo K=8	28.18%	65.45%	60.38%	76.33%	70.08%	42.92%	39.40%	54.68%
3.1B	DiLoCo K=16	27.38%	63.03%	59.27%	75.95%	66.41%	41.72%	39.60%	53.34%
3.1B	MuLoCo K=1	37.10%	68.60%	63.54%	77.69%	71.42%	42.58%	42.60%	57.65%
3.1B	MuLoCo K=2	37.52%	68.35%	64.17%	77.69%	72.18%	42.24%	41.80%	57.71%
3.1B	MuLoCo K=4	35.43%	67.49%	61.01%	77.75%	70.20%	41.72%	43.40%	56.71%
3.1B	MuLoCo K=8	35.29%	66.82%	60.85%	77.31%	70.45%	43.00%	41.40%	56.45%
3.1B	MuLoCo K=16	29.50%	65.46%	60.38%	75.90%	68.01%	39.68%	42.40%	54.48%
15B	DP AdamW	57.20%	77.69%	71.19%	81.39%	78.11%	52.30%	45.40%	66.18%
15B	DP Muon	59.14%	79.05%	71.19%	81.12%	77.23%	53.07%	46.80%	66.80%
15B	DiLoCo K=1	59.80%	79.06%	71.43%	81.34%	77.99%	52.56%	46.40%	66.94%
15B	DiLoCo K=16	54.15%	77.59%	69.30%	81.72%	77.65%	52.56%	46.80%	65.68%
15B	MuLoCo K=1	59.49%	79.06%	71.11%	81.61%	78.11%	52.73%	46.60%	66.96%
15B	MuLoCo K=16	56.96%	77.18%	70.72%	80.74%	76.60%	53.16%	46.20%	65.94%

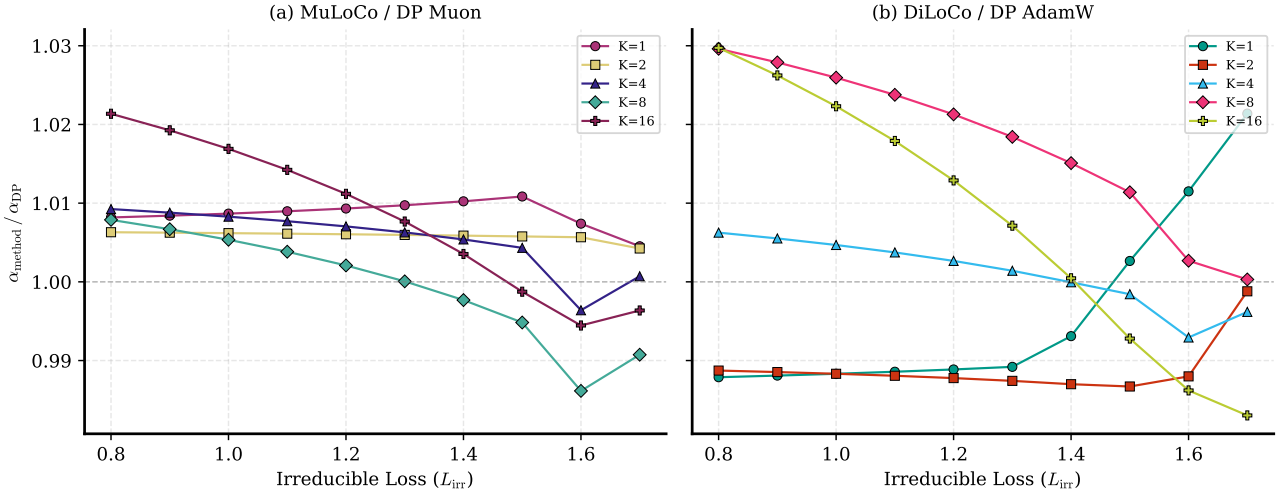


Figure 16. **Scaling exponents as a function of the assumed irreducible loss.** We fit compute scaling laws of the form $L(C) = aC^\alpha + L_{\text{irr}}$ to our 150M-3.1B datapoints while holding L_{irr} fixed to a predetermined value that is shared across all methods. For each L_{irr} , we report the resulting scaling exponent α normalized by the corresponding data-parallel (DP) baseline (y-axis: $\alpha_{\text{method}}/\alpha_{\text{DP}}$), so values above 1 indicate better than DP scaling while values below 1 indicate worse scaling than DP. The left panel shows MuLoCo normalized by DP Muon, while the right panel shows DiLoCo normalized by DP AdamW. At lower irreducible losses, the fits suggest that higher-worker MuLoCo configurations (larger K) become increasingly competitive and will catch up to and surpass DP performance at scale.

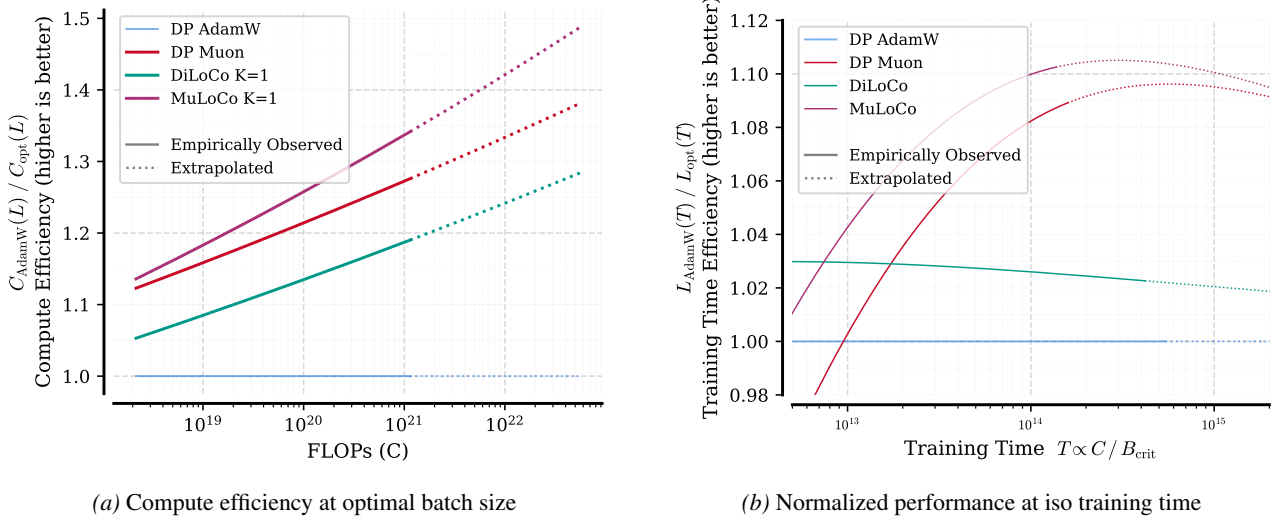


Figure 17. MuLoCo improves both compute efficiency and training-time-normalized performance. (a) We report iso-loss compute efficiency, $C_{\text{AdamW}}(L)/C_{\text{opt}}(L)$, using scaling laws fit at each method’s optimal batch size. Values above one indicate that optimizer m reaches the same loss as AdamW using fewer FLOPs. (b) We report normalized performance at fixed training time, where training time is measured as FLOPs divided by the critical batch size. This metric is evaluated at B_{crit} and assumes enough accelerators are scaled in proportion to the critical batch size to realize the corresponding reduction in sequential computation. Under this assumption, MuLoCo achieves an $\sim 8\text{--}10\%$ loss improvement over AdamW for the same training-time budget. Solid curves indicate the empirically observed regime, while dotted curves indicate extrapolation beyond the range of available data.

Compute efficiency and training-time-normalized performance. Figure 17 provides two complementary views of the efficiency gains implied by our scaling laws. The compute-efficiency plot uses the same compute–loss scaling-law methodology as Sec. E.2, fit across the extensively swept 150M–3.1B models, but evaluates each optimizer at its optimal batch size. For a target loss L , we invert the fitted loss curve for each optimizer to obtain the compute required to reach that loss, $C_m(L)$, and report $C_{\text{AdamW}}(L)/C_{\text{opt}}(L)$. Values above one, therefore, indicate that optimizer m reaches the same loss using fewer FLOPs than AdamW. In contrast, the training-time-normalized performance plot evaluates each method at its critical batch size, using $T \propto C/B_{\text{crit}}(C)$ as a proxy for the amount of sequential computation required. This asks a different question: given the same training-time budget, how much better is the final loss if enough accelerators are used in parallel to scale with the method’s critical batch size? We find that MuLoCo improves over AdamW under both views: it is more compute-efficient at the optimal batch size ($\sim 1.35\times$), and when training at B_{crit} , it can achieve an $\sim 8\text{--}10\%$ loss improvement over AdamW for the same training-time budget.

Extrapolating the Critical Batch Size. Given that the critical batch size is known to scale with training data (Bergsma et al., 2025; Zhang et al., 2025), we fit power laws $B_{\text{crit}}(D) = aD^\alpha$ to extrapolate the critical batch size (B_{crit}) as training data (D) is increased. With B_{opt} being the optimal batch size, we take B_{crit} to be the largest batch size such that $L(B_{\text{crit}}) \leq 0.01L(B_{\text{opt}})$. We exclude MuLoCo and DiLoCo at $K \neq 1$ in these experiments as their batch sizes were not sufficiently swept to determine B_{crit} at each scale. The power laws are fit using L-BFGS as described in the previous section. Figure 19 (a) plots the power law fit for Muon DP, AdamW DP, MuLoCo K=1, and DiLoCo K=1. We observe that Muon DP and MuLoCo have much larger scaling exponents than DiLoCo and AdamW DP. This demonstrates that their critical batch sizes will grow with scale at a faster rate. We also find that MuLoCo and DiLoCo both have larger prefactors than their data-parallel baselines, meaning they will have a slightly larger critical batch size at each scale.

Finding 11: MuLoCo K=1 is projected to have larger critical batch sizes than DP Muon at scale.

Training time efficiency at iso-loss. The largest-scale pre-training runs often take over a month of training time to complete. They are primarily bottlenecked by the number of sequential gradient descent steps required for strong performance, which is directly determined by batch size used. While the strongest possible final performance is achieved at the optimal batch size (B_{opt}), to maintain reasonable training time costs for the largest runs, practitioners generally train at the critical batch size (B_{crit}), since with enough training data $B_{\text{crit}} \gg B_{\text{opt}}$. Therefore, optimizers should not only be compared w.r.t their best possible performance but also based on their training time to loss at B_{crit} . A reasonable proxy for training time is $T \propto C/B_{\text{crit}}(C)$, the amount of FLOPs that must be computed sequentially when training at B_{crit} (Bergsma et al., 2025).

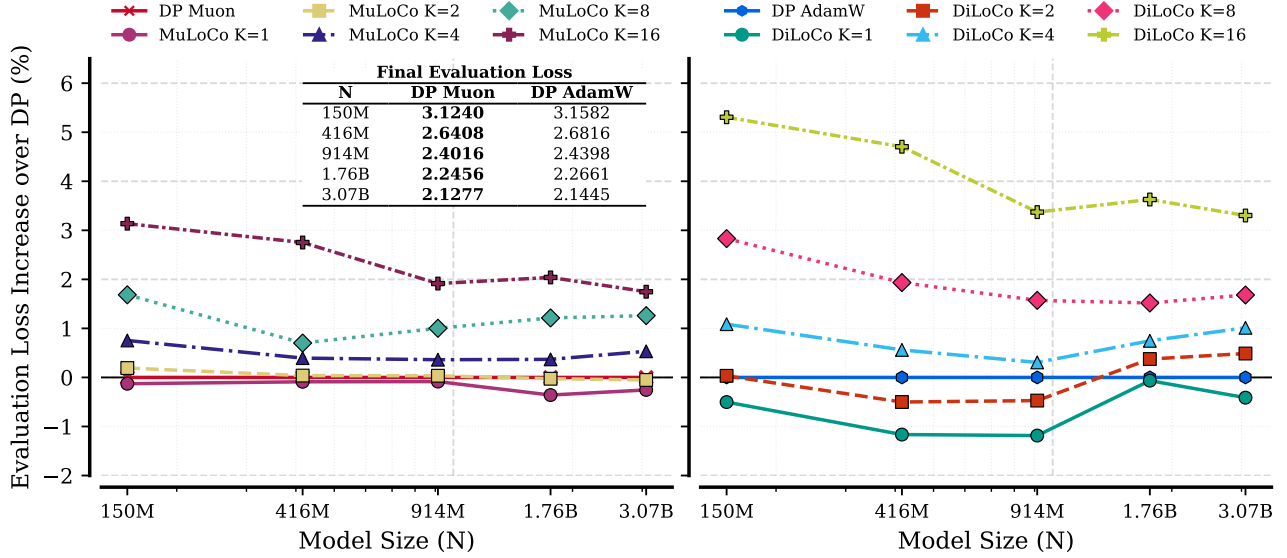


Figure 18. **Relative performance vs. data-parallel baselines as model size and worker count scale.** We report the percent increase in final evaluation loss over data-parallel (DP) for MuLoCo (left) and DiLoCo (right) across model sizes and worker counts $K \in \{1, 2, 4, 8, 16\}$. MuLoCo’s performance relative to DP improves at larger worker counts, and we observe the relative gap shrinks with scale for $K = 16$ workers. Note that MuLoCo is normalized by *Muon* DP, while DiLoCo is normalized by *AdamW* DP.

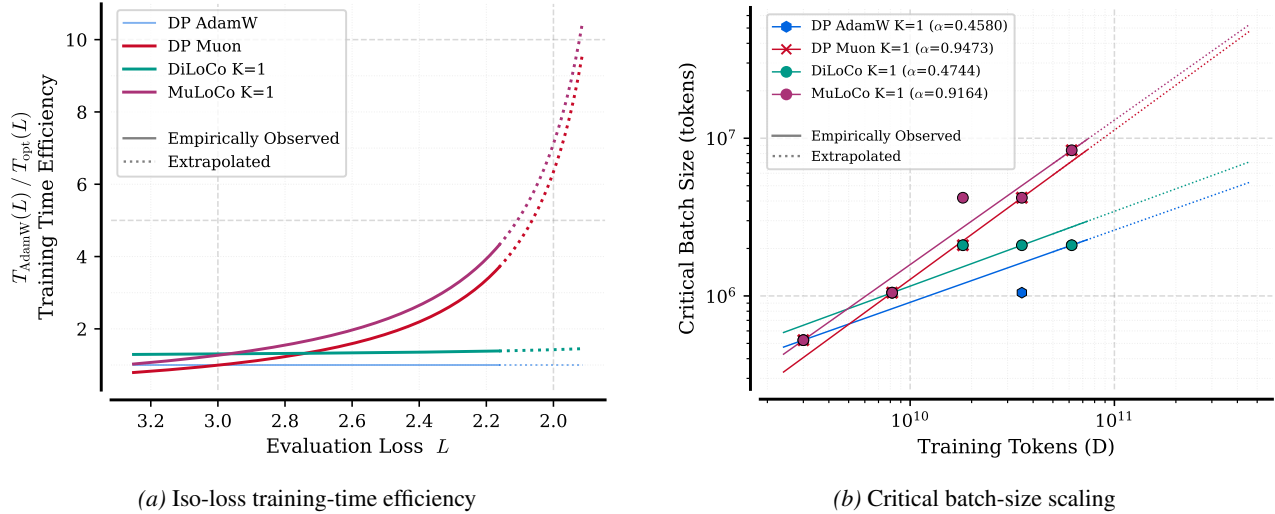


Figure 19. **MuLoCo achieves substantial training-time gains at iso-loss due to its large critical batch sizes.** (left) Reports the iso-loss training-time efficiency of different optimizers relative to DP AdamW. The ratio $T_{\text{AdamW}}(L)/T_{\text{opt}}(L)$ measures how many more sequential FLOPs AdamW requires to reach the same loss L , assuming sufficient parallel and well-interconnected accelerators to realize the speedup from larger critical batch sizes. At observed scales, MuLoCo reaches the same loss as AdamW in roughly $\sim 4.5\times$ fewer sequential FLOPs. (right) Power-law fits of the critical batch size (CBS) versus training tokens show that MuLoCo’s CBS is larger than Muon data-parallel (DP) and significantly larger than AdamW DP and DiLoCo, indicating a wider regime of near-lossless batch scaling. Dotted curves indicate extrapolation beyond the range of available data.

In Figure 19 (a), we directly compare the iso-loss training-time efficiency of different optimizers relative to DP AdamW, finding that $K=1$ MuLoCo has the strongest training-time advantage.

To accomplish this, for each optimizer m we fit two scaling laws from our training runs: a loss curve $L_m(C) = a_m C^{b_m} + L_0$ with a shared irreducible loss L_0 across all $K=1$ methods, and a critical batch size scaling law $B_{\text{crit},m}(D) = A_m D^{\alpha_m}$, where D denotes training tokens and we use the Chinchilla-optimal relationship $D = 20N$, $C = 6ND$. Given a target

loss L , we invert the loss scaling law to obtain the compute $C_m(L)$ required by each optimizer, and then evaluate the corresponding training-time proxy $T_m(L) = C_m(L)/B_{\text{crit},m}(C_m(L))$. The iso-loss training-time efficiency is then defined as $T_{\text{AdamW}}(L)/T_{\text{opt}}(L)$: a value greater than one indicates that optimizer m reaches the same loss as AdamW in fewer sequential FLOPs.

This metric captures the practical advantage of optimizers with larger critical batch sizes: they can absorb more data-parallel workers without diminishing returns, training faster in wall-clock time for a fixed target loss. Equivalently, the iso-loss training-time ratio decomposes into a compute-efficiency term and a parallelism term:

$$\frac{T_{\text{AdamW}}(L)}{T_{\text{opt}}(L)} = \underbrace{\frac{C_{\text{AdamW}}(L)}{C_{\text{opt}}(L)}}_{\text{compute savings}} \times \underbrace{\frac{B_{\text{crit}}^{(m)}(C_m(L))}{B_{\text{crit}}^{(\text{AdamW})}(C_{\text{AdamW}}(L))}}_{\text{parallelism advantage}}. \quad (8)$$

We find that K=1 MuLoCo reaches the same loss as DP AdamW in roughly $\sim 4.5\times$ fewer sequential steps at observed scales. This advantage is much larger than its compute-only savings because MuLoCo combines improved compute scaling with a substantially larger critical batch size.

Finding 12: At iso-loss, K=1 MuLoCo reaches the same loss as DP AdamW in roughly $\sim 5\times$ fewer sequential steps at observed scales.

E.4. Extended 15B results

In this section, we use the datapoints from our scaling study to extrapolate optimal hyperparameters to 15B scale, train with these hyperparameters, and evaluate the performance of the resulting models. While reporting the optimal performance is not guaranteed for each method since we did not sweep at this scale, these experiments provide us with an understanding of the performance attainable from different optimizers, an understanding of how large batch training and communication overhead contributed to wall-clock training time, and an increased confidence in the scalability of our method.

Extrapolating optimal hyperparameters. Using the data collected in our scaling experiments, we fit three new power laws: (1) $\lambda(N) = aN^\alpha$ (weight decay), (2) $\eta(N) = aN^\alpha$ (learning rate), and (3) $B_{\text{opt}}(D) = aD^\alpha$ (optimal batch size). Following our tuning protocol from section 4, the weight decay power law is fit to the optimal value found at $B = 1\text{M}$ and extrapolated in this regime. The optimal value found is subsequently rescaled according to the extrapolated optimal batch size. For learning rate and batch size we simply fit power laws to their optimal values for each optimizer. It should be noted that our power laws for with B_{opt} were fit with $K=1$ MuLoCo, having $B_{\text{opt}} = 4\text{M}$ at 3.1B scale. However, further sweeping later revealed that this was a local minimum (see Fig. 6). This means that $K=1$ MuLoCo’s batch size used at 15B scale is likely closer to the B_{crit} than B_{opt} and that $K=1$ MuLoCo’s optimal final performance is likely slightly under-reported. Further details in addition to the optimal values used are reported in F.2 of the appendix.

Table 8. **Zero-shot LM Evaluation Results for $K=1$ and $K=16$ optimizers at 15B scale.** We extrapolate optimal hyperparameters for $K=1$ and $K=16$ optimizers and train 15B parameter models with them. We report accuracy for MMLU, WinoGrande, and normalized accuracy for HellaSwag, PIQA, ARC-E, ARC-C, OBQA.

Model Size	Optimizer	Eval. Loss	MMLU	HellaSwag	WinoGrande	PIQA	ARC-E	ARC-C	OBQA	Mean Acc.
15B	DP AdamW	1.8873	57.20%	77.69%	71.19%	81.39%	78.11%	52.30%	45.40%	66.18%
15B	DP Muon	1.8642	59.14%	79.05%	71.19%	81.12%	77.23%	53.07%	46.80%	66.80%
15B	DiLoCo $K=1$	1.8914	59.80%	79.06%	71.43%	81.34%	77.99%	52.56%	46.40%	66.94%
15B	MuLoCo $K=1$	1.8839	59.49%	79.06%	71.11%	81.61%	78.11%	52.73%	46.60%	66.96%
15B	DiLoCo $K=16$	1.9062	54.15%	77.59%	69.30%	81.72%	77.65%	52.56%	46.80%	65.68%
15B	MuLoCo $K=16$	1.9168	56.96%	77.18%	70.72%	80.74%	76.60%	53.16%	46.20%	65.94%

Evaluating final performance. Table 8 reports the final loss values obtained for models trained with different optimizers in addition to their zero-shot performance across MMLU, HellaSwag, WinoGrande, PIQA, ARC-E, ARC-C, and OBQA. Across optimizers, we find that all methods converge to *nearly* the same average downstream accuracy, with MuLoCo ($K=1$), DiLoCo ($K=1$), and data-parallel Muon performing marginally stronger, while the $K=16$ variants and DP AdamW perform slightly worse. Importantly, as shown in the per-step view of Figure 21, MuLoCo ($K=1$) trains at the largest batch size (17M) while reaching nearly identical final evaluation loss and evaluation performance to DiLoCo ($K=1$) and DP Muon both trained for many more steps. This demonstrates that MuLoCo $K=1$ can leverage more parallelism without sacrificing performance. A similar advantage holds for MuLoCo ($K=16$), which reached comparable performance to DiLoCo ($K=16$) and DP AdamW while also having a larger batch size than both.

Evaluating wall-clock training time when accounting for batch size, optimizer step times, and communication overhead. We also collected system-level metrics during training, enabling wall-clock analyses under networking constraints (Figure 22). In bandwidth-constrained regimes (10 Gbit/s), the communication-efficient optimizers dominate: both $K=16$ methods train substantially faster than $K=1$ MuLoCo, despite their smaller batch sizes, with MuLoCo ($K=16$) emerging as the fastest configuration due to its larger extrapolated optimal batch size. As bandwidth increases (800 to 12,800 Gbit/s), communication becomes less of a bottleneck and leveraging more compute in parallel becomes more important. In the high bandwidth regime, the ability of MuLoCo ($K=1$) to leverage very large batch sizes becomes advantageous, leading it ultimately train the fastest.

Finding 13: When training under bandwidth constraints, $K=16$ MuLoCo has the fastest wall-clock training time, while $K=1$ MuLoCo is fastest in high-bandwidth settings.

Additional training time results. First we present timings for the 15B training runs with Muon and AdamW (Table 9). We find that the additional overhead of Muon’s optimizer step is negligible at 15B scale. Using these metrics, we provide idealized wall-clock training estimates for different bandwidth networks, assuming the GPUs are scaled proportionally to batch size (Table 10 and Figures 22). Finally, we also present validation loss curves for these training runs as a function of training tokens and training steps (Figure 21).

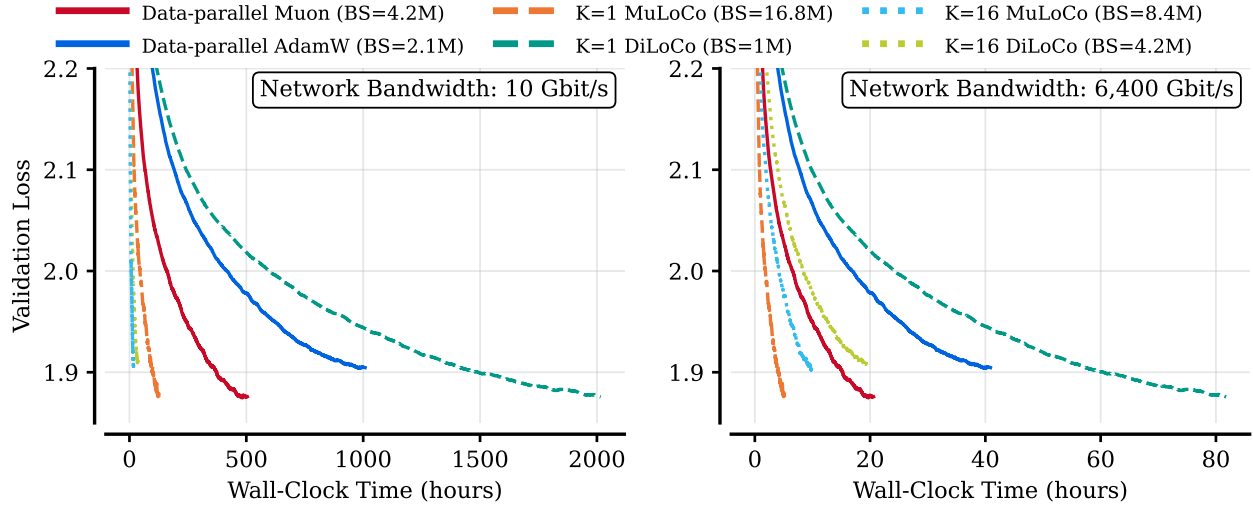


Figure 20. Idealized wall-clock training time for 15B runs under low- and high-bandwidth networking. We estimate training time by combining (i) network communication time, (ii) optimizer step time, and (iii) forward/backward (FW/BW) compute time. The left panel reports validation loss versus wall-clock time assuming a low-bandwidth network, while the right panel reports the same quantities in a datacenter setting with high bandwidth. In the low-bandwidth regime, **K=16 MuLoCo** achieves the fastest time-to-loss and dominates the DiLoCo baselines; in the datacenter regime, **K=1 MuLoCo** achieves the fastest time-to-loss. In both cases, MuLoCo’s larger batch sizes translate into faster wall-clock training to comparable loss than corresponding DiLoCo configurations.

Table 9. System-level performance comparison of training with AdamW and Muon at 15B scale. End-to-end step time when training at local batch size 2 and global batch size 1024 with 16 FSDP ranks and 32 data parallel ranks. Metrics correspond to the median over the first 500 steps. Relative Δ computed as $\frac{\text{Muon} - \text{AdamW}}{\text{AdamW}} \times 100$. We observe that the additional overhead of Muon’s optimizer step is negligible at 15B scale.

Metric	AdamW	Muon	Relative Δ (%)
End-to-End (s)	0.9753	0.9832	+0.81
MFU (%)	39.89	39.57	−0.80
TFLOPS/GPU	394.54	391.37	−0.80
Throughput (tokens/s/GPU)	4,200	4,166	−0.81
Memory Complexity (Parameter copies)	4	3	−25

Table 10. Idealized wall-clock training time under bandwidth constraints. We report idealized wall-clock times (in hours) estimated using the 15B system measurements in 9. The estimates factor in global batch size, communication overhead, optimizer step time, and forward/backward (FW/BW) compute time. We observe that communication-efficient optimizers dominate at low bandwidth, while larger batch sizes dominate under high bandwidth.

Method	Training Time (hours)					
	10 Gbit/s	100 Gbit/s	400 Gbit/s	1,600 Gbit/s	3,200 Gbit/s	6,400 Gbit/s
Data-parallel AdamW (BS=2.1M)	1008.2	136.3	63.6	45.4	42.4	40.9
Data-parallel Muon (BS=4.2M)	504.3	68.3	32.0	22.9	21.4	20.6
K=1 DiLoCo (BS=1M)	2016.3	272.5	127.2	90.8	84.8	81.8
K=1 MuLoCo (BS=16.8M)	126.1	17.1	8.0	5.7	5.3	5.2
K=16 DiLoCo (BS=4.2M)	35.8	21.3	20.1	19.8	19.7	19.7
K=16 MuLoCo (BS=8.4M)	18.0	10.7	10.1	10.0	9.9	9.9

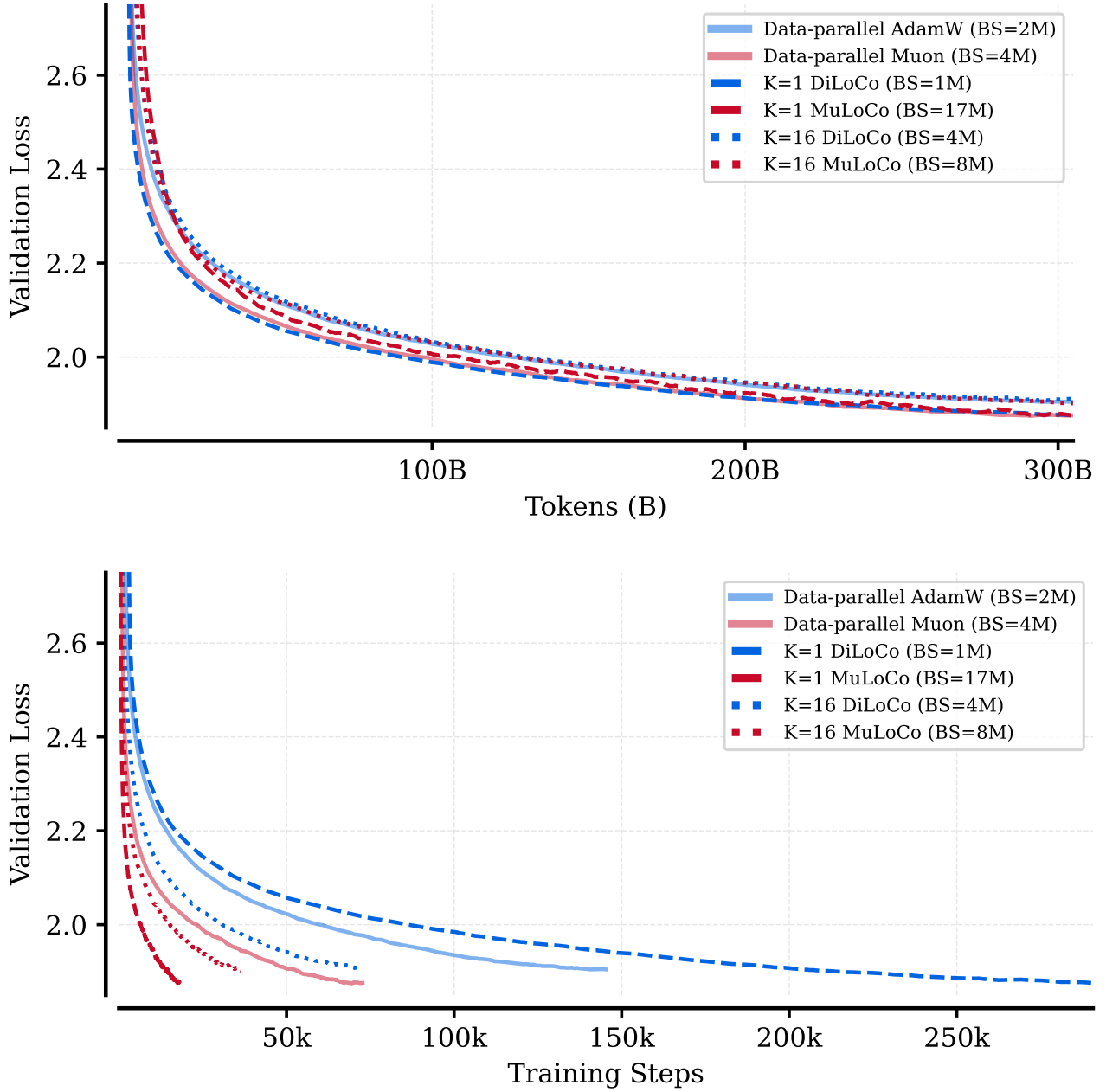


Figure 21. **Validation loss for 15B pre-training runs.** We extrapolate optimal hyperparameters to 15B scale and train models with different optimizers. *Top:* validation loss as a function of training tokens. *Bottom:* validation loss as a function of optimization steps. Despite using a $16\times$ larger batch size, MuLoCo ($K = 1$, BS=17M) reaches nearly the same final loss as DiLoCo ($K = 1$, BS=1M), demonstrating its strong capability for large-batch training. For $K = 16$, DiLoCo narrows the gap to data-parallel AdamW, but both remain notably worse than data-parallel Muon, MuLoCo $K=1$, and DiLoCo $K=1$. Despite reaching nearly the same loss as DP AdamW and DiLoCo $K=16$, MuLoCo $K = 16$ can leverage larger batch sizes.

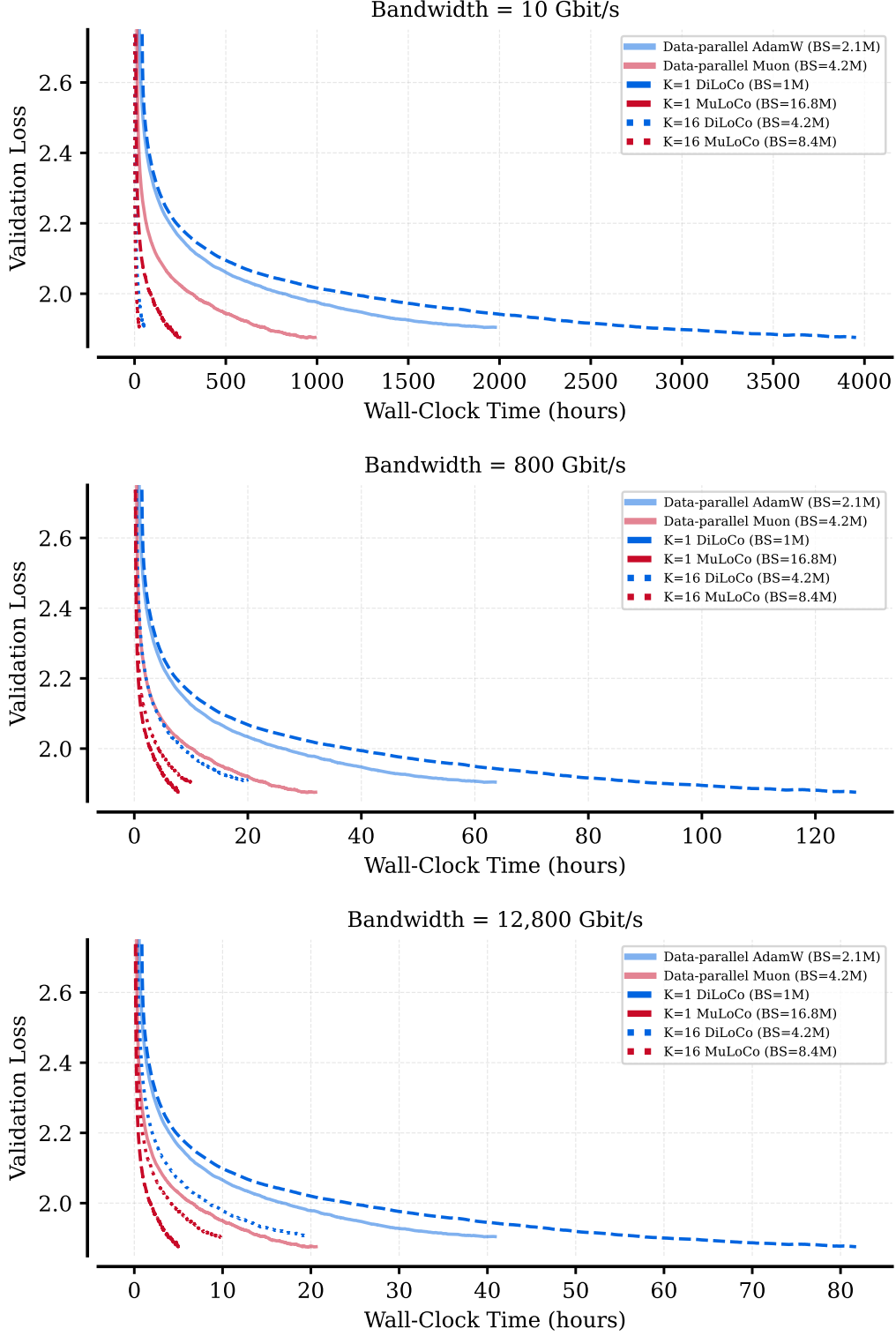


Figure 22. Training time under bandwidth constraints. We compare validation loss versus wall-clock time across network bandwidths of 10, 800, and 12,800 Gbit/s under the assumption that the cluster size is scaled proportional to the batch size. Both batch size and communication time critically influence overall wall-clock training time. In the bandwidth-constrained regime (10 Gbit/s), batch size still matters, but reducing communication frequency dominates: the $K=16$ optimizers train substantially faster than $K=1$ MuLoCo despite using smaller batch sizes. As network speed increases, communication becomes less of a bottleneck and larger batch sizes become more advantageous. At 12,800 Gbit/s, we observe that $K=1$ MuLoCo (with its larger batch size) trains faster, illustrating the shifting trade-off between communication cost and statistical efficiency.

Table 11. Hyperparameter search spaces. Ranges are aggregated across all model scales and worker counts. They correspond to the ranges that we needed to sweep in order to ensure the optimal HP value was an interior point of the values tried. Learning rates follow a $\sqrt{2}$ grid; batch sizes follow a powers-of-2 grid. Weight decay is rescaled with batch size per Wang & Aitchison (2024). Outer optimizer HPs are swept at 416M and reused at larger scales.

Hyperparameter	DP AdamW	DP Muon	DiLoCo	MuLoCo
Inner LR (η_{in})	[1.7e-4, 6.3e-2]	[3.9e-3, 3.5e-1]	[3.5e-4, 8.8e-2]	[4.9e-4, 1.8e-1]
Weight decay (λ)	[6.3e-5, 1e-1]	[1e-4, 1e-1]	[1.6e-5, 1.7e-1]	[5e-5, 8e-2]
Global batch size (B)	{32–4096}	{32–8192}	{8–8192}	{8–8192}
Outer LR (η_{out})	—	—	{0.4–1.1}	{0.4–1.1}
Outer momentum (μ)	—	—	{0.5–0.9}	{0.5–0.9}
Total runs (all scales)	339	260	822	808

F. Optimal hyperparameters

F.1. Hyperparameter Sweep Details

As described in Section 4, we conduct extensive hyperparameter sweeps at each model scale for all optimizers and worker counts. In total, our scaling study comprises over 2,200 individual pre-training runs across six model scales (150M–15B), four optimizers (DP AdamW, DP Muon, DiLoCo, MuLoCo), and five worker counts ($K \in \{1, 2, 4, 8, 16\}$). We summarize the sweep ranges, tuning strategy, and optimal hyperparameters found below.

Sweep ranges. Table 11 summarizes the hyperparameter search spaces used across our scaling study. All learning rates are swept on a multiplicative grid of powers of $\sqrt{2}$ (i.e., each value is $\sqrt{2} \times$ the previous), while batch sizes are swept on powers of 2. Weight decay values are rescaled according to Wang & Aitchison (2024) as the batch size changes: for DP methods we rescale by the global batch size, and for DiLoCo/MuLoCo we rescale by the per-worker batch size (B/K). The outer optimizer hyperparameters (η_{out}, μ) are extensively swept at the 416M scale and reused at other scales, following Charles et al. (2025)’s finding that they remain roughly constant across model sizes.

Hyperparameter tuning. At all model scales, we tune the weight decay (λ), inner learning rate (η_{in}), global batch size (B), outer learning rate (η_{out}), and outer momentum (μ) according to the following procedure.

1. (Data-Parallel λ) We first perform a grid search across λ within $\{1^{-1}, 1^{-2}, 1^{-3}, 1^{-4}\}$ and η_{in} values that are integer powers of $\sqrt{2}$ for the AdamW and Muon data-parallel baselines. We fix the $B = 1\text{M}$ at all scales for this initial sweep.
2. (Data-Parallel η_{in}, B) Having established the optimal λ^* value at $B = 1\text{M}$, we now perform a grid-search over integer powers of 2 for B and integer powers of $\sqrt{2}$ for η_{in} , while rescaling λ^* according to Wang & Aitchison (2024) as B is varied.
3. (DiLoCo/MuLoCo $\lambda, \eta_{\text{in}}, B$) Re-using the existing λ^* found above for $B = 1\text{M}$, at each worker count $K \in \{1, 2, 4, 8, 16\}$ we perform a grid-search over powers of 2 B and powers of $\sqrt{2}$ η_{in} , while rescaling λ^* according to Wang & Aitchison (2024) using the per-worker batch size (B/K).
4. (DiLoCo/MuLoCo η_{out}, μ) When sweeping the outer optimizer’s hyperparameters, we follow empirical guidance from Charles et al. (2025) who found that for DiLoCo they remain roughly constant across model sizes. As such, we perform an extensive grid search at the 416M scale sweeping across $\lambda, \eta_{\text{in}}, \eta_{\text{out}}, \mu$, and B . During this extensive grid search, we tune η_{out} and μ by searching within $\{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$ with other hyperparameters spanning ranges from above. We report the optimal μ and η_{out} values found in Figure 23 and re-use them at larger model scales.

Optimal hyperparameters at each scale. Tables 12–14 report the optimal hyperparameters found at each model scale. Several trends emerge:

- **Learning rate decreases with scale** for AdamW-based methods (DP AdamW, DiLoCo), dropping roughly $30\times$ from 150M to 3.1B. In contrast, Muon-based methods (DP Muon, MuLoCo) maintain relatively stable learning rates across scales.

Table 12. **Optimal hyperparameters for data-parallel baselines** at each model scale. η_{in} : inner learning rate; λ : weight decay; B : global batch size (sequences); B_{tok} : global token batch size ($= B \times 2048$). Since $K = 1$ for DP methods, $B/K = B$.

Scale	Method	η_{in}	λ	B	B_{tok}
150M	DP AdamW	4.42e-2	1.25e-4	64	128K
	DP Muon	1.25e-1	1.25e-3	64	128K
416M	DP AdamW	1.10e-2	3.00e-2	512	1M
	DP Muon	8.84e-2	2.50e-3	512	1M
914M	DP AdamW	5.52e-3	5.00e-3	256	512K
	DP Muon	3.13e-2	5.00e-3	256	512K
1.76B	DP AdamW	1.38e-3	1.00e-1	512	1M
	DP Muon	3.13e-2	1.00e-3	512	1M
3.07B	DP AdamW	1.38e-3	5.00e-4	256	512K
	DP Muon	2.21e-2	1.00e-3	512	1M
15.2B	DP AdamW	1.73e-4	2.00e-4	1024	2M
	DP Muon	1.10e-2	4.00e-4	2048	4M

- **Weight decay is tied to batch size** via the rescaling rule of Wang & Aitchison (2024). Within a fixed scale, λ decreases as K increases because the per-worker batch size (B/K) decreases.
- **Batch size grows with scale** for all methods. DP methods and $K=1$ workers generally use $B = 32$ – 256 at 150M–3.1B. At higher K , the per-worker batch size decreases while the total token batch size ($B \times K \times 2048$) remains roughly constant within a scale.
- **MuLoCo favors lower outer momentum at small K.** As shown in Figure 23, MuLoCo uses $\mu = 0.6$ at $K = 1$ versus $\mu = 0.8$ for DiLoCo. At $K \geq 4$, both converge to $\mu = 0.8$ – 0.9 .
- **Outer learning rate increases with K** for both DiLoCo and MuLoCo, rising from $\eta_{\text{out}} = 0.6$ – 0.7 at $K = 1$ to $\eta_{\text{out}} = 1.0$ at $K = 16$.

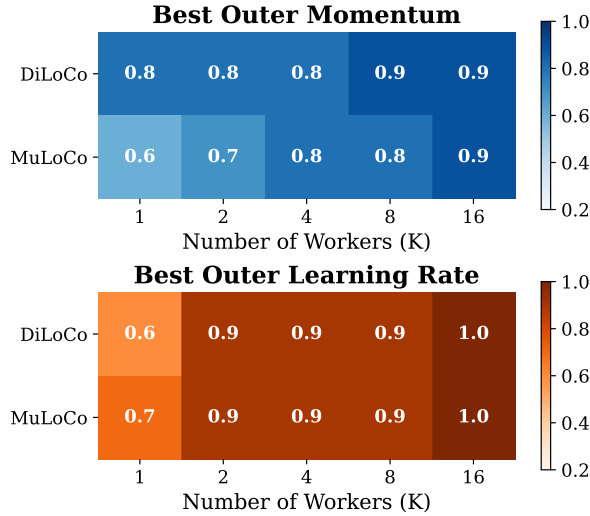


Figure 23. **At low worker counts MuLoCo’s outer momentum should be decreased.** When tuning the outer optimizer hyperparameters for MuLoCo and DiLoCo, we found that MuLoCo favors lower momentum coefficients at lower worker counts relative to DiLoCo.

Outer optimizer hyperparameters. For both DiLoCo and MuLoCo, we observe a consistent pattern across all model sizes: (1) the outer learning rate increases from 0.6–0.7 at $K = 1$ to 1.0 at $K = 16$, and (2) the outer momentum increases from 0.6–0.8 at $K = 1$ to 0.9 at $K \geq 8$. Notably, MuLoCo uses a lower outer momentum ($\mu = 0.6$) than DiLoCo ($\mu = 0.8$)

Table 13. Optimal hyperparameters for DiLoCo. At each model scale and worker count K , B/K : per-worker batch size (sequences); B : global batch size; B_{tok} : global token batch size ($= B \times 2048$). All runs use $H = 30$ synchronization steps and SGD with Nesterov momentum as the outer optimizer.

Scale	K	η_{in}	λ	B/K	B	B_{tok}	η_{out}	μ
150M	1	2.21e-2	2.50e-4	128	128	256K	0.6	0.8
	2	4.42e-2	1.25e-4	64	128	256K	0.9	0.8
	4	2.21e-2	6.25e-5	32	128	256K	0.9	0.8
	8	4.42e-2	3.13e-5	16	128	256K	0.9	0.9
	16	2.21e-2	3.13e-5	16	256	512K	1.0	0.9
416M	1	1.10e-2	5.00e-3	256	256	512K	0.6	0.8
	2	1.10e-2	2.50e-3	128	256	512K	0.9	0.8
	4	1.10e-2	1.25e-3	64	256	512K	0.9	0.8
	8	7.81e-3	1.25e-3	64	512	1M	0.7	0.9
	16	2.76e-3	6.25e-4	32	512	1M	1.0	0.9
914M	1	5.52e-3	5.00e-3	256	256	512K	0.6	0.8
	2	3.91e-3	2.50e-3	128	256	512K	0.9	0.8
	4	3.91e-3	1.25e-3	64	256	512K	0.9	0.8
	8	2.76e-3	1.25e-3	64	512	1M	0.9	0.9
	16	1.95e-3	6.25e-4	32	512	1M	1.0	0.9
1.76B	1	1.95e-3	1.00e-3	512	512	1M	0.6	0.8
	2	1.38e-3	5.00e-4	256	512	1M	0.9	0.8
	4	1.95e-3	2.50e-4	128	512	1M	0.9	0.8
	8	1.38e-3	1.25e-4	64	512	1M	0.9	0.9
	16	1.38e-3	6.25e-5	32	512	1M	1.0	0.9
3.07B	1	1.38e-3	5.00e-4	256	256	512K	0.6	0.8
	2	6.91e-4	5.00e-4	256	512	1M	0.9	0.8
	4	1.38e-3	2.50e-4	128	512	1M	0.9	0.8
	8	1.38e-3	1.25e-4	64	512	1M	0.9	0.9
	16	6.91e-4	1.25e-4	64	1024	2M	1.0	0.9

at $K = 1$, which is consistent with Muon’s optimizer steps already being well-aligned (see Section 5 paragraph 2), reducing the benefit of additional momentum. These values are extensively tuned at the 416M scale (Table 11) and reused at all other scales. For a visualization of optimal values, see Figure 23

Weight decay rescaling. At each model scale, we first determine the optimal weight decay λ^* for DP AdamW and DP Muon at a fixed batch size of $B = 1\text{M}$ tokens (approximately $B = 512$ sequences). When sweeping other batch sizes, we rescale λ^* following Wang & Aitchison (2024). For DiLoCo and MuLoCo, the weight decay is rescaled according to the per-worker batch size (B/K), which explains the systematic decrease of λ with increasing K observed in Tables 13 and 14.

Table 14. **Optimal hyperparameters for MuLoCo.** At each model scale and worker count K . B/K : per-worker batch size (sequences); B : global batch size; B_{tok} : global token batch size ($= B \times 2048$). All runs use $H = 30$ synchronization steps and SGD with Nesterov momentum as the outer optimizer.

Scale	K	η_{in}	λ	B/K	B	B_{tok}	η_{out}	μ
150M	1	6.25e-2	1.25e-3	64	64	128K	0.7	0.6
	2	8.84e-2	1.25e-3	64	128	256K	0.9	0.7
	4	8.84e-2	6.25e-4	32	128	256K	0.9	0.8
	8	1.25e-1	6.25e-4	32	256	512K	0.9	0.8
	16	6.25e-2	3.13e-4	16	256	512K	1.0	0.9
416M	1	4.42e-2	5.00e-3	256	256	512K	0.7	0.6
	2	4.42e-2	5.00e-3	256	512	1M	0.9	0.7
	4	4.42e-2	2.50e-3	128	512	1M	0.9	0.8
	8	6.25e-2	1.25e-3	64	512	1M	0.9	0.8
	16	4.42e-2	6.25e-4	32	512	1M	1.0	0.9
914M	1	2.21e-2	5.00e-3	256	256	512K	0.7	0.6
	2	2.21e-2	2.50e-3	128	256	512K	0.9	0.7
	4	2.21e-2	1.25e-3	64	256	512K	0.9	0.8
	8	3.13e-2	1.25e-3	64	512	1M	0.9	0.8
	16	3.13e-2	1.25e-3	64	1024	2M	1.0	0.9
1.76B	1	3.13e-2	2.00e-3	1024	1024	2M	0.7	0.6
	2	2.21e-2	5.00e-4	256	512	1M	0.9	0.7
	4	2.21e-2	2.50e-4	128	512	1M	0.9	0.8
	8	1.56e-2	1.25e-4	64	512	1M	0.9	0.8
	16	1.56e-2	1.25e-4	64	1024	2M	1.0	0.9
3.07B	1	1.56e-2	5.00e-4	256	256	512K	0.7	0.6
	2	1.56e-2	5.00e-4	256	512	1M	0.9	0.8
	4	1.56e-2	2.50e-4	128	512	1M	0.9	0.8
	8	1.56e-2	2.50e-4	128	1024	2M	0.9	0.8
	16	1.56e-2	2.50e-4	128	2048	4M	1.0	0.9

F.2. Extrapolating HPs to 15B scale

At the 15B scale, running full hyperparameter sweeps is computationally prohibitive. Instead, we extrapolate optimal weight decay, inner learning rate, and batch size from power law fits to smaller scales. To promote better accelerator utilization, we use the nearest power-of-2 batch size to the optimal extrapolated value. Depending on whether we rounded the batch size up or down, we adjust the learning rate up or down, respectively to its nearest $\sqrt{2}$ power. For weight decay, we used $1e-4$ for both AdamW and Muon at $B = 1\text{M}$ since the extrapolated values were close enough, and this matches our other experiments. The hyperparameters used are reported in table 15. It should be noted that our power laws with B^* were fit with K=1 MuLoCo, having $B^* = 4\text{M}$ at 3.1B scale. However, further sweeping later revealed that this was a local minimum (see Fig. 6). This means that K=1 MuLoCo’s batch size is likely closer to the CBS than B^* at 15B scale and that its optimal final performance is likely slightly under-reported. The power law fits used for B^* and η are reported in Figure. 24.

Table 15. **Hyperparameters for 15B runs.** These are extrapolated from power law fits to smaller scales (150M–3.07B) with minor manual adjustments. B/K : per-worker batch size (sequences); B : global batch size (sequences); B_{tok} : global token batch size ($= B \times 2048$).

Method	K	η_{in}	λ	B/K	B	B_{tok}	η_{out}	μ
DP AdamW	1	1.73e-4	2.00e-4	1024	1024	2M	—	—
DP Muon	1	1.10e-2	4.00e-4	2048	2048	4M	—	—
DiLoCo	1	3.45e-4	1.00e-4	512	512	1M	0.6	0.8
DiLoCo	16	3.45e-4	2.50e-5	128	2048	4M	1.0	0.9
MuLoCo	1	1.56e-2	1.60e-3	8192	8192	16M	0.7	0.6
MuLoCo	16	1.10e-2	5.00e-5	256	4096	8M	1.0	0.9

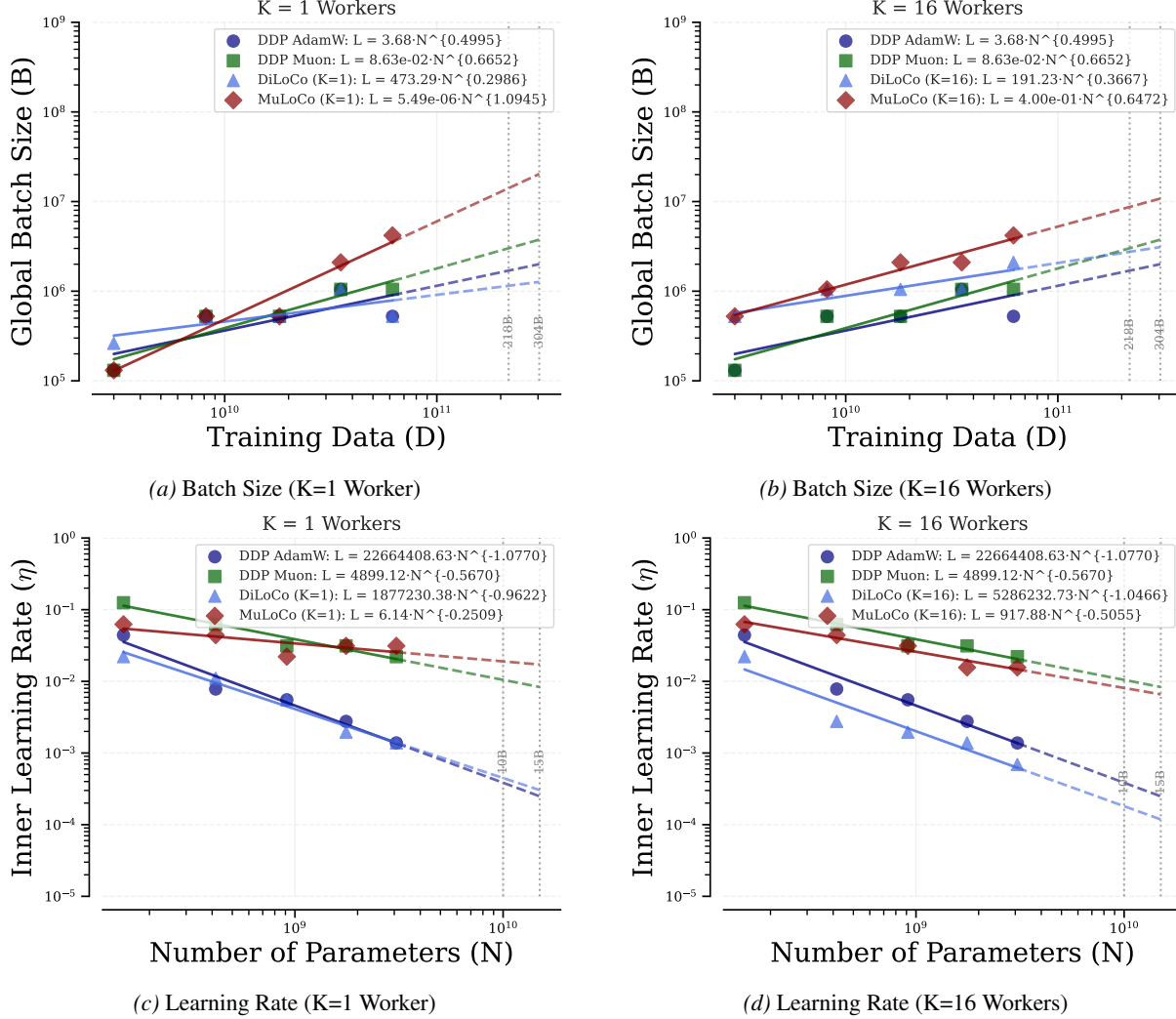


Figure 24. **Extrapolating optimal batch size (B^*) and inner learning rate (η_{in}) for different optimization algorithms.** We fit a power law in tokens (D) to the optimal batch size and in parameters (N) for the optimal learning rates at each scale. ****NOTE 1:** These were fit before realizing that $B=4M$ was a local minimum for K=1 MuLoCo at 3.1B scale, so the optimal batch size is slightly larger. We still chose to report these fits since they were used in our 15B experiments. ****NOTE 2:** The legend in these plots shows an powerlaw with misleading variables. It should be $B = AD^\alpha$ for (a,b) $\eta_{in} = AN^\alpha$ for (c,d). The y and x axes are correct.

G. A Robust Evaluation Loss Estimate

Throughout this work, we evaluate each training run by computing a smoothed evaluation loss $\hat{L}$ from its validation loss trajectory. Directly using the final recorded validation loss is unreliable for comparing runs and fitting scaling laws, as validation losses are logged at discrete intervals and exhibit non-negligible noise, overestimating or underestimating final performance (see Figure 25). This is likely caused by sampling different final evaluations depending on the number of training iterations for the run in question. We therefore adopt a time-weighted exponential moving average (EMA) that accounts for the temporal spacing between measurements and provides a smooth, robust estimate of the final performance.

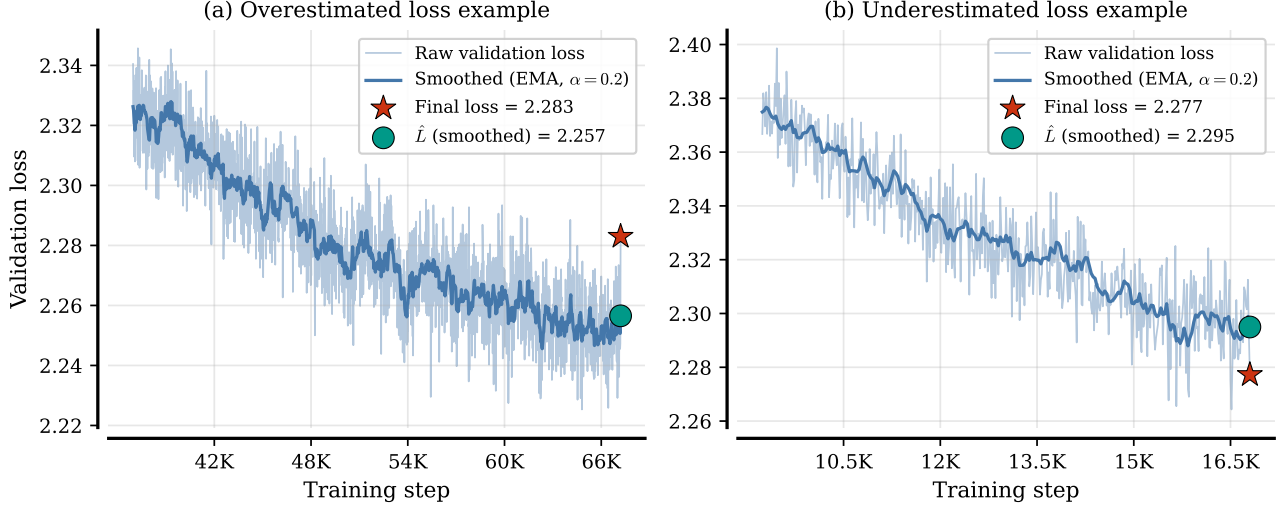


Figure 25. **Motivation for reporting a smoothed evaluation loss $\hat{L}$ rather than the final logged validation loss.** Each panel shows a real validation-loss trajectory near the end of training (raw points) together with the time-weighted EMA used in Sec. G (with $\alpha = 0.2$ and filtering to synchronization boundaries at $H = 30$). **Left:** an *overestimated* final loss, where the last evaluated batch happens to be unusually difficult (the final logged loss exceeds $\hat{L}$). **Right:** an *underestimated* final loss, where the last evaluated batch is unusually easy (the final logged loss is below $\hat{L}$). These examples illustrate that directly using the final recorded validation loss can introduce noise-driven bias in run comparisons and scaling-law fits, whereas $\hat{L}$ provides a more robust final performance estimate.

Concretely, each run records a validation loss trajectory $\{(\ell_i, t_i)\}_{i=1}^N$, where ℓ_i is the validation loss (cross-entropy on 2M token batches from the NemotronCC held-out split) measured at training step t_i . Since DiLoCo and MuLoCo synchronize worker parameters every H inner steps, we first filter the trajectory to retain only measurements taken at synchronization boundaries, i.e., steps t_i satisfying $t_i \bmod H = 0$ (with $H = 30$ throughout this work). This filtering ensures that all methods—including data-parallel baselines—are evaluated at steps where the model parameters correspond to a post-synchronization state rather than an intermediate local state. For data-parallel methods, which effectively synchronize every step, this subsampling still produces a dense set of measurements.

After filtering, we obtain a subsequence $\{(\ell_j, t_j)\}_{j=1}^M$ and apply a time-weighted EMA. Unlike a standard EMA that assigns a fixed weight α to each new observation, we adjust the effective smoothing factor based on the elapsed time $\Delta t_j = t_j - t_{j-1}$ between consecutive measurements. The smoothed sequence $\{s_j\}_{j=1}^M$ is computed as

$$s_1 = \ell_1, \quad s_j = \tilde{\alpha}_j \ell_j + (1 - \tilde{\alpha}_j) s_{j-1}, \quad j = 2, \dots, M, \quad (9)$$

where the adaptive smoothing coefficient is

$$\tilde{\alpha}_j = 1 - \exp\left(-\frac{\alpha \Delta t_j}{H}\right). \quad (10)$$

Here $\alpha > 0$ is a base smoothing parameter. When measurements are evenly spaced at every H steps ($\Delta t_j = H$), the adaptive coefficient reduces to the constant $\tilde{\alpha} = 1 - e^{-\alpha}$. The evaluation loss for the run is then $\hat{L} = s_M$, the final value of the smoothed sequence.

We use $\alpha = 0.2$ throughout, which yields $\tilde{\alpha} \approx 0.181$ at the nominal spacing of $H = 30$ steps. This provides substantial smoothing (an effective window spanning roughly the last $1/\tilde{\alpha} \approx 5$ –6 synchronization intervals) while remaining responsive

enough to track the loss trajectory near convergence. All scaling law fits, hyperparameter comparisons, and optimal run selections reported in this paper use this smoothed evaluation loss $\hat{L}$.